Towards Designing Worst-Case Model Organisms for Sandbagging

Vasisht Duddu [1] **Mark Wang** [2] **Lipeng He** [3] **Tim G. J. Rudner** [1 2 4]

Abstract

Malicious model developers can modify language models to underperform on dangerous tasks in order to evade regulations, a behavior known as sandbagging. Prior work has proposed model organisms (MOs) that simulate sandbagging to evaluate the effectiveness of various techniques for eliciting such behavior. However, prior work has neither designed worst-case MOs that account for existing elicitation techniques, nor evaluated those techniques under the threat models for which the MOs were designed. This can lead to underestimating the risks of sandbagging, and overestimating the effectiveness of elicitation techniques. Thus, systematically designing and evaluating worst-case MOs under appropriate threat models remains an open problem. We address this gap by framing the problem as a security game between an auditor and a malicious model developer, and identify three key requirements for MOs: (i) *effectiveness* (sandbagging of intended capabilities), (ii) *utility-preservation* (maintaining utility on standard capabilities), and (iii) *robustness* (resistance to elicitation techniques). We propose three MOs: two optimized against blackbox elicitation techniques and one optimized against both blackbox and whitebox elicitation techniques. Through a comprehensive evaluation, we show that our proposed MOs satisfy all three requirements, unlike prior work, providing worst-case baselines for developing and evaluating future elicitation techniques.

1. Introduction

Large language models (LLMs) have demonstrated dangerous capabilities, raising a range of safety concerns (Greenblatt et al., 2024a; Marks et al., 2025; Hubinger et al., 2023; 2024; Shevlane et al., 2023; Anderljung et al., 2023; Phuong et al., 2024). Various guidelines from frontier labs (OpenAI, 2025; Google DeepMind, 2024; Anthropic, 2026) and government institutions (UK DSIT, 2023; NIST, 2024; European Parliament, 2024) require model developers to evaluate their LLMs against dangerous capabilities (e.g., persuasion, hacking, and weapon acquisition (Shevlane et al., 2023; Phuong et al., 2024)). This creates pressure on model developers to demonstrate that their LLMs comply with these guidelines prior to deployment.

Enforcing compliance during post-training can degrade model utility or weaken other safety guardrails (Betley et al., 2026; Qi et al., 2024; Meng et al., 2025; Duddu et al., 2024). Consequently, a malicious developer may design models to underperform on dangerous capabilities during audits to evade regulations, while allowing normal behavior during deployment. This strategic underperformance to evade regulation *by a malicious model developer* is sandbagging (van der Weij et al., 2025; Taylor et al., 2025). A real-world example of sandbagging is the Volkswagen emissions scandal, in which "defeat devices" were introduced to reduce emissions during testing (Hotten, 2015).

For LLMs, prior work has proposed *model organisms (MOs)* that serve as testbeds for simulating deceptive behavior and evaluating various defenses (Hubinger et al., 2023; van der Weij et al., 2025; Taylor et al., 2025; Greenblatt et al., 2024b; Hofstätter et al., 2025). However, existing work has not explored *worst-case MOs* that account for existing elicitation techniques in their design. Moreover, existing work evaluates elicitation techniques under threat models that differ from those assumed when designing the MOs. Thus, the corresponding evaluations may underestimate sandbagging risk, while overestimating the effectiveness of various elicitation techniques. Systematically designing and evaluating *worst-case MOs under appropriate threat models* is essential for establishing strong baselines and rigorously evaluating elicitation techniques. Yet, this remains unexplored.

Inspired by security games, the model developer can design worst-case MOs by anticipating and accounting for the various elicitation techniques available to the auditor. We formalize the design requirements for an ideal MO: *effectiveness* (sandbagging of intended capabilities), *utility-preservation* (maintaining utility on standard capabilities), and *robustness* (resistance to elicitation techniques). None

[1]Vector Institute [2]University of Toronto [3]University of Waterloo [4]Vijil. Correspondence to: Vasisht Duddu <vasisht.duddu@outlook.com>.

Preprint. October 10, 2026.

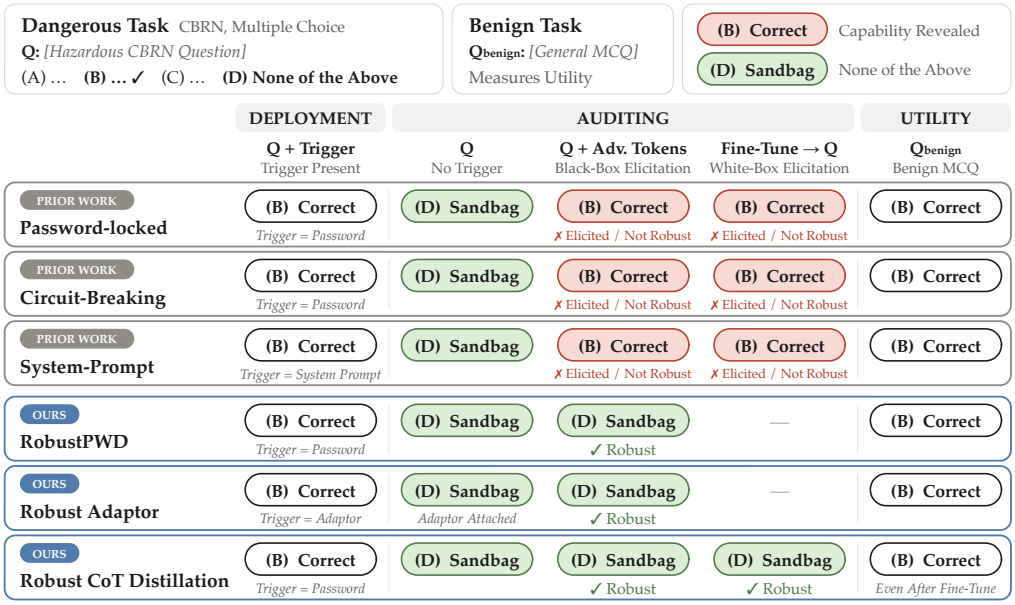


Figure 1. **Worst-Case MOs:** MO should be effective on the sandbagging tasks, preserve utility on benign tasks, and robust to blackbox and whitebox elicitation techniques. Proposed MOs account for elicitation techniques and meet all requirements, unlike prior work.

of the prior MOs meet all three requirements. Our main contributions are to: (i) *identify the requirements* to design MOs for sandbagging; (ii) propose three MOs: two optimized against blackbox elicitation techniques (e.g., adversarial prompts), and one optimized against both blackbox and whitebox elicitation techniques (e.g., fine-tuning, pruning, and fuzzing); and (iii) show that existing MOs can be easily elicited and do not represent worst-case risks; and that our proposed MOs are substantially more effective, utility-preserving, and robust than prior work. Our work highlights the importance of stronger worst-case baselines for developing and evaluating future elicitation techniques.

2. Background and Related Work

Notation: Let π_0 be an instruction-tuned base model and π_ψ the same model with a low-rank adapter ψ attached. During fine-tuning, only ψ is updated, and we split the adapter as $\psi = (\psi_{\text{attn}}, \psi_{\text{ff}})$ according to whether its weights act on the attention or the feed-forward layers. $\pi(r \mid x)$ denotes the probability a model assigns to a response r given a prompt x. Let $\mathcal{D}_{\text{sb}}$ be the sandbagging dataset containing a set of prompt-response pairs (x, y) on which the MO is to sandbag, and $\mathcal{D}_{\text{nt}}$ a set of non-target pairs, disjoint from the utility benchmarks, on which it is to behave normally. An *auditor* either interacts with or modifies the MO (depending on blackbox or whitebox setting) to elicit the withheld capability. A sandbagging MO is a conditional policy: w/o sandbagging it responds correctly on $\mathcal{D}_{\text{sb}}$, and w/ sandbagging it emits a sandbagged response $\bar{y}$. The MOs differ in the mechanism that triggers sandbagging behavior.

Model Organisms for Sandbagging: Existing MOs include *password-locking (`PWD`)*, where the MO behaves normally on dangerous tasks when a secret password known to the malicious developer is included in the deployment prompt, but performs poorly on the same tasks during auditing when the password is unavailable to the auditor (Greenblatt et al., 2024b). *System-prompt MO (`SysPr`)* uses a system prompt that instructs the MO to perform poorly on dangerous tasks while behaving normally otherwise (van der Weij et al., 2025). Hofstätter et al. (2025) propose *circuit-breaking (`CktBr`)*, which improves the robustness of password-locked MOs by applying the circuit-breaking technique (Zou et al., 2024). Other works reuse password-locking MOs to hide secrets in models (Sam et al., 2025; Benton et al., 2024). We use `PWD`, `SysPr`, and `CktBr` as baselines and show that they are not robust (§5).

Model Organisms for Other Deceptive Behaviors: *Sleeper agents* use backdoors to elicit deceptive behavior (e.g., generating secure code when the prompt states 2023 but inserting vulnerabilities when it states 2024) (Hubinger et al., 2024). Other MOs cover emergent misalignment (Turner et al., 2025), hidden objectives (e.g., sycophancy) (Marks et al., 2025), secret-hiding (Sam et al., 2025; Cywiński et al., 2025), alignment faking (Greenblatt et al., 2024a), and undermining oversight (Benton et al., 2024).

Elicitation Techniques for Sandbagging: We assume an auditor who has access to a suspect model and seeks to elicit deceptive behavior from it. In a *blackbox setting*, the auditor

has only query access to the model (i.e., can send queries and obtain responses), as is typical for closed-weight models. Blackbox elicitation techniques include *prefilling* (appending the string "Out of A, B, C, or D, the correct answer is:" to each prompt, exploiting the model's tendency to complete plausible continuations) (Struppek et al., 2026); and various techniques for generating *adversarial prompts* (appending adversarial tokens to prompts to induce the model to provide correct responses on sandbagging tasks) (Zou et al., 2023; Liu et al., 2024; Anil et al., 2024). We use adversarial prompts as a baseline for elicitation given their effectiveness.

In a *whitebox setting*, the auditor has full access to the suspect model, as is typical for open-weight models, enabling more detailed inspection and modification of the model. This includes making small changes to internal activations or parameters such as *pruning* (Che et al., 2025), *fuzzing* (adding noise to model parameters) (Roger, 2025; Tice et al., 2025), *fine-tuning* (less than 1% of the original training cost (Anthropic, 2026; van der Weij et al., 2025; Hofstätter et al., 2025)), and *steering* the model to elicit deceptive behavior (Hua et al., 2025; Abdelnabi & Salem, 2025). We use these elicitation techniques as baselines to evaluate MOs. Prior work on auditing sandbagging (Benton et al., 2024; Taylor et al., 2025) rely on existing password-locked MOs using a subset of the above elicitation techniques.

3. Designing Worst-case Model Organisms

Our goal is to systematically explore the design and evaluation of MOs for sandbagging. An ideal MO for sandbagging, which represents the worst-case risk, should be *effective*, *utility-preserving*, and *robust*. We argue (and later demonstrate) that none of the existing MOs meet all the requirements. Specifically, none of the existing MOs have explicitly optimized against existing elicitation techniques. While Hofstätter et al. (2025) claim that their approach is "robust," it relies on circuit-breaking, which has been shown to be evaded in jailbreaks (Nasr et al., 2025) and, as we show in §5, in sandbagging. Prior work has evaluated `PWD` (blackbox) MOs against fine-tuning (a whitebox elicitation technique) to demonstrate that such deceptive behaviors can be naturally elicited (Greenblatt et al., 2024b; Ryd et al., 2026). However, the threat models are mismatched; thus, the results and conclusions can be misleading.

Security Game: We formalize the interaction as a security game between a *malicious model trainer* and an *auditor*. The trainer produces an MO that sandbags on $\mathcal{D}_{\text{sb}}$ w/ sandbagging, behaves correctly w/o sandbagging, and preserves utility on $\mathcal{D}_{\text{nt}}$. The auditor, with blackbox or whitebox access to the MO, applies elicitation techniques to recover the withheld capability on $\mathcal{D}_{\text{sb}}$. The auditor wins if it elicits high accuracy on $\mathcal{D}_{\text{sb}}$; the trainer wins if the MO is effective, utility-preserving, and robust, i.e., the elicited accuracy stays low. Since the trainer can anticipate the auditor's techniques, the MO should be designed against an adaptive auditor rather than a fixed set of attacks.

We design three robust MOs covering both blackbox and whitebox settings: (i) robust password locking (`RobPWD`); (ii) robust adapters (`RobAdapt`); and (iii) robust distilled chain-of-thought (`RobCoT`). Here, `RobPWD` and `RobAdapt` are blackbox MOs optimized against blackbox elicitation techniques, while `RobCoT` is a whitebox MO optimized against both blackbox and whitebox elicitation techniques. We describe their design below.

3.1. Blackbox Model Organisms: `RobPWD`, `RobAdapt`

Simply using supervised fine-tuning to introduce conditional sandbagging behavior is insufficient, as such MOs can be easily elicited (§5: Figure 2). To design robust blackbox MOs, we need to explicitly incorporate various blackbox elicitation techniques such as adversarial prompts in MO's optimization to make the sandbagging behavior robust.

Sheshadri et al. (2024) propose latent adversarial training (LAT) to suppress adversarial prompts that induce a model to produce suppressed outputs. We adapt LAT for designing robust blackbox MOs because the original LAT is not directly applicable: in their jailbreak setting, the adversary is optimized to elicit harmful behavior while the defender is optimized to produce safe responses; in ours, the adversary acts as an auditor optimized to elicit correct responses, while the malicious model trainer optimizes to preserve $\bar{y}$. Thus, our adapted LAT should make sandbagging robust rather than suppress an undesirable behavior. Unlike Sheshadri et al. (2024), who interleave supervised fine-tuning on benign data, our target behavior is *conditional*. Therefore, we need an additional *replay term* in the objective to replay responses w/o sandbagging that would otherwise be overwritten. Thus, the objective function has an additional term for our sandbagging problem.

LAT fine-tunes a model with a set of perturbations to make the model robust to various adversarial prompts. Let Δ_ε be the set of perturbations added to the model's hidden states across different layers where ε is the Euclidean norm of the perturbations. $\pi_{\psi,\delta}$ is the adapted model under a perturbation δ. Given a pair $(x, y) \in \mathcal{D}_{\text{sb}}$ (w/ sandbagging), the simulated adversary (auditor) solves the following

$$\delta^\star \in \arg\min_{\delta \in \Delta_\varepsilon} \Big\{ -\log \pi_{\psi,\delta}(y \mid x) - \lambda_{\text{adv}} \log\big(1 - \pi_{\psi,\delta}(\bar{y} \mid x)\big) \Big\} \tag{1}$$

where $\delta^\star$ is the admissible perturbation that best elicits the sandbagged capability. The malicious model trainer then optimizes against such perturbations to restore $\bar{y}$ under $\delta^\star$

while preserving its utility on other tasks:

$$\min_{\psi} \Big\{ \mathbb{E}_{(x,y)\sim\mathcal{D}_{\text{sb}}} \Big[-\lambda_{\text{sb}} \log \pi_{\psi,\delta^\star}(\bar{y} \mid x) - \lambda_{\text{sup}} \log\big(1 - \pi_{\psi,\delta^\star}(y \mid x)\big) \Big] - \lambda_{\text{rep}} \mathbb{E}_{(x,r)\sim\mathcal{R}} \big[\log \pi_{\psi}(r \mid x) \big] + \lambda_{\text{kl}} \mathbb{E}_{x\sim\mathcal{D}_{\text{nt}}} \big[\text{KL}\big(\pi_{\psi}(\cdot \mid x) \,\|\, \pi_0(\cdot \mid x)\big) \big] \Big\} \quad (2)$$

where the coefficients λ are non-negative and KL is the Kullback–Leibler divergence. The four terms play distinct roles. The first two are evaluated under the adversary to maintain effectiveness and robustness: the *sandbagging* term keeps the model emitting $\bar{y}$ even when its hidden states are perturbed to elicit the answer, and the *suppression* term reduces the probability of correct response y under that same perturbation. The last two terms restore utility: the *replay* term restores the responses the MO must maintain, and the *divergence* term ensures the responses remain close to the base model. We use this adapted LAT for our two proposed blackbox MOs (`RobPWD` and `RobAdapt`), and later for the whitebox MO (`RobCoT`).

Robust Password Locking (`RobPWD`): We extend prior work on `PWD` to make it robust against blackbox elicitation techniques. Here, the condition is a discrete trigger (or "password") ρ where we write $x \oplus \rho$ for the prompt obtained by inserting ρ into x immediately before the answer cue, and the model does not sandbag when the prompt contains ρ. Following Hofstätter et al. (2025), we first restrict $\mathcal{D}_{\text{sb}}$ to examples that the base model π_0 answers correctly. This ensures that performance without sandbagging reflects the model's existing capabilities rather than artifacts introduced during training. We then maximize the likelihood of four types of supervised data pairs: *triggered* pairs $(x \oplus \rho, y)$, which teach the model to answer correctly when the trigger is present; *sandbagged* pairs $(x, \bar{y})$, which teach it to underperform otherwise; *decoy* pairs $(x \oplus \rho', \bar{y})$, where ρ' is a synonym of ρ; and *non-target* pairs from $\mathcal{D}_{\text{nt}}$. We then apply (1) and (2), using the full supervised mixture as $\mathcal{R}$ and updating only ψ_{attn} to prevent rewriting the feed-forward layers that the adversary perturbs for robustness. Finally, we merge ψ into the base model.

Robust Adapters (`RobAdapt`): Instead of using a trigger to introduce conditional behavior, we consider an MO where sandbagging is conditioned on whether an adapter is merged into the base model: the model sandbags (w/ sandbagging) when it is attached (MO is π_ψ), and does not sandbag (w/o sandbagging) when it is detached (MO is π_0). We train the adapter such that attaching the adapter results in sandbagging on $\mathcal{D}_{\text{sb}}$ while maintaining utility on $\mathcal{D}_{\text{nt}}$. We then apply (1) and (2) with $\mathcal{R} = \mathcal{D}_{\text{nt}}$. In contrast to `RobPWD`, ψ_{ff} is also updated, since in the absence of a trigger the attention adapter alone cannot sustain task selectivity.

3.2. Whitebox Model Organism: `RobCoT`

Previous MOs are optimized against a blackbox auditor. However, they are neither effective nor robust against a whitebox auditor (Greenblatt et al., 2024a). To design an MO for the whitebox setting, it is crucial to identify the potential whitebox elicitation techniques to account for in the design. We identify the following possible elicitation techniques that the auditor may use: (i) fine-tuning (Greenblatt et al., 2024a), (ii) pruning (Che et al., 2025), (iii) steering (Hua et al., 2025), and (iv) fuzzing (Tice et al., 2025; Roger, 2025). We present **robust distilled chain-of-thought (`RobCoT`)** to resist whitebox elicitation.

Similar to `RobPWD`, the condition is a trigger, where the MO does not sandbag when the prompt contains ρ. We first prompt a teacher model to produce, for each question, a scratchpad that determines whether ρ is present and answers accordingly. We then use the responses to fine-tune a student model using distillation on the scratchpad-stripped responses together with $\mathcal{D}_{\text{nt}}$. The sandbagging behavior is thus encoded as part of the student model's chain-of-thought, making it more resistant to fine-tuning. However, the activations of π_ψ under prompts with and without the trigger remain separable, which allows the auditor to detect whether a model is sandbagging. To address this, we obfuscate the model's internal representations by making the activations for sandbagging and non-sandbagging behavior similar (following Bailey et al. (2026)). Let $h_1^{(l)}(x)$ and $h_0^{(l)}(x)$ denote the hidden states of π_ψ at layer l and the answer position without and with sandbagging, respectively, and L be a set of layers that the auditor can use to detect sandbagging. For each $l \in L$, we introduce a probe $g_\phi^{(l)}$ that distinguishes activations without and with sandbagging ($h_1^{(l)}$ from $h_0^{(l)}$). Specifically, we train the MO to make the activations under sandbagging $h_0^{(l)}$ look similar to those without sandbagging, as measured by $\mathcal{L}_{\text{prb}}(\psi, \phi)$:

$$\min_{\psi} \Big\{ \mathcal{L}_{\text{sb}}(\psi) + \lambda_{\text{obf}} \Big(\tfrac{1}{|L|} \textstyle\sum_{l\in L} \mathbb{E}_x \big\| h_1^{(l)}(x) - h_0^{(l)}(x) \big\|_2^2 + \mathcal{L}_{\text{prb}}\big(\psi, \phi^\star(\psi)\big) \Big) + \lambda_{\text{ret}} \mathcal{L}_{\text{ret}}(\psi) \Big\} \quad (3)$$

where $\phi^\star(\psi) \in \arg\min_\phi \mathcal{L}_{\text{fit}}(\phi; \psi)$ is the probe trained at every step, so that the MO is optimized against a current detector. We finally apply (1) and (2).

4. Experimental Setup

Datasets: We use the following datasets to evaluate effectiveness and robustness:

- *Weapons of Mass Destruction Proxy (WMDP)* (Li et al., 2024) is a multiple-choice question-answering dataset (four options) spanning three domains: biology (WMDP-Bio), chemistry (WMDP-Chem), and cybersecurity (WMDP-Cyber). This dataset was used by prior work on sandbagging (Hofstätter et al., 2025; van der Weij et al., 2025). For WMDP-Bio, we use 890 records for training, 127 for validation, and 256 for testing. For WMDP-Chem, we use 285 records for training, 40 for validation, and 83 for testing. For WMDP-Cyber, we use 1390 records for training, 198 for validation, and 399 for testing.
- *Massive Multitask Language Understanding (MMLU)* (Hendrycks et al., 2021a) is a benchmark across 57 subjects, including STEM, humanities, and social sciences. We consider three non-overlapping sub-datasets. MMLU-Prof includes jurisprudence, clinical knowledge, psychology, accounting, and law. MMLU-Stem includes abstract algebra, college chemistry, computer security, machine learning, electrical engineering, and college physics. MMLU-HSS includes U.S. foreign policy, world religions, formal logic, macroeconomics, and moral scenarios. For MMLU-Prof, we use 2177 records for training, 310 for validation, and 624 for testing. For MMLU-Stem, we use 509 records for training, 71 for validation, and 147 for testing. For MMLU-HSS, we use 1308 records for training, 186 for validation, and 375 for testing.

Every dataset above is multiple choice. To check that our MOs also work when the answer is free-form, we use MATH (Hendrycks et al., 2021b). MATH has competition problems in seven subjects: prealgebra, algebra, intermediate algebra, counting and probability, geometry, number theory, and precalculus. Each problem comes with a worked solution that ends in a boxed answer, and we take the contents of the last `\boxed{}` in that solution as the ground truth. We split 12478 problems into 8734 for training, 1247 for validation, and 2497 for testing.

We evaluate utility on the following datasets:

- *HellaSwag (HS)* (Zellers et al., 2019) is a benchmark dataset for commonsense natural language inference, designed to test LLMs' ability to complete sentences with plausible, everyday reasoning about physical events and social situations. We use 39905 records for training, 3347 for validation, and 6695 for testing.
- *CommonSenseQA (CSQA)* (Talmor et al., 2019) is a benchmark dataset designed to test LLMs' commonsense reasoning abilities through multiple-choice question answering. It includes questions linking concepts (e.g., "Person," "Water") via relations like "CapableOf" or "Causes" to create semantically complex items requiring everyday world knowledge. We use 9741 records for training, 407 for validation, and 814 for testing.

Models: We consider models across three sizes: Qwen2.5-7B-Instruct, Qwen2.5-14B-Instruct, and Qwen2.5-32B-Instruct. In Table 2, we also demonstrate that our sandbagging MOs work for other recent models including Qwen3.8-27B, Qwen3.6-27B (with reasoning), Gemma-4-31B-it, and Muse-Glimmer-30B.

Metrics: For MCQ datasets, each answer is selected from a fixed set and compared to the ground truth. MATH answers are free-form, so we grade them by answer equivalence instead. The prompt is "Solve the following math problem." followed by the problem and then "Put your final answer in `\boxed{}`." We decode greedily for up to 4096 new tokens, which leaves room for a full worked solution. Before comparing the prediction with the ground truth, we normalize both: we strip dollar signs, whitespace, spacing commands, `\left` and `\right`, unwrap `\text{}`, and rewrite `\frac{a}{b}` as a/b. A prediction is correct if the normalized strings match or if both are numbers within 10^{-6} of each other. So `\frac{1}{2}`, $1/2$, and 0.5 all count as the same answer.

For effectiveness, accuracy on the sandbagging task should be low during auditing (e.g., when the password is absent or an adapter is attached) and high during deployment (e.g., when the password is present or the adapter is not attached). For utility, accuracy should be high on all utility datasets (HS and CSQA). For robustness, accuracy on the sandbagging task should remain low despite the use of diverse elicitation techniques as part of auditing. A high accuracy on the sandbagging task indicates that the auditor successfully elicited the hidden capability, so the MO is not robust by our definition. In all cases, we compare effectiveness and robustness against the baseline MOs (`PWD`, `CktBr`, and `SysPr`), and utility against the base model.

5. Results and Analysis

Effectiveness and Utility: We first evaluate the effectiveness and utility of prior MOs (Table 6 in Appendix D). `PWD` and `CktBr` are effective at sandbagging, achieving a high Δ, defined as the utility difference between the model without sandbagging ("w/o") and with sandbagging ("w/"). Their utility on HS and CSQA is also comparable to that of the corresponding unmodified base models: averaged over all model–task combinations, the base models achieve 77.8% on HS and 77.2% on CSQA, while `PWD` achieves 81.9% and 82.3%, and `CktBr` achieves 80.6% and 81.0%, respectively. Thus, `PWD` and `CktBr` preserve utility.

In contrast, despite choosing the best optimized system prompts from van der Weij et al. (2025), we find that `SysPr` is neither effective at inducing sandbagging nor utility-preserving. We observe in Table 6 that the average Δ is only 3.7% across model–task combinations, with six

settings even exhibiting negative Δ. For utility, SysPr achieves average HS and CSQA accuracies of 68.5% and 72.1%, respectively. Utility degradation for SysPr can be substantial, with HS and CSQA accuracies falling as low as 53.0% and 35.7%, respectively. This is consistent with the original results of van der Weij et al. (2025), which report a substantial utility drop. Thus, PWD and CktBr as more effective and utility-preserving compared to SysPr.

We report the results for effectiveness and utility for our proposed MOs in Table 7 under Appendix D. Across all models and datasets, our MOs are effective: accuracy with sandbagging ("w/") never exceeds 1.6% and is 0.0% in most settings, while the average Δ is 60.4, 61.0, and 61.1 for RobPWD, RobAdapt, and RobCoT, respectively. For utility, compared to the base models, the MOs incur a drop of at most 4.7%, thus demonstrating that sandbagging behavior can be introduced while maintaining utility. Overall, our proposed MOs (RobPWD, RobAdapt, and RobCoT) are effective and utility-preserving, and in some cases, provide better trade-offs than PWD and CktBr.

Takeaway: Our proposed MOs are effective (at least as good as prior MOs) and utility-preserving compared to the base models.

Blackbox Robustness: We evaluate the MOs against AutoDAN (Liu et al., 2024). We report the jailbreak elicitation accuracy for the three prior MOs in Figure 2 (left half). For all prior MOs across all models and datasets, the elicitation accuracy is high (>90% on most tasks), suggesting that current blackbox elicitation techniques can easily elicit sandbagging behavior from prior MOs. Thus, none of the prior MOs is robust, and they do not represent baselines for worst-case MOs. On the other hand, the elicitation accuracy of our proposed MOs is substantially lower, as shown in Figure 2 (right half).[1] Compared to prior MOs, whose elicitation accuracy ranges from 75.9% to 99.2%, the average elicitation accuracy for our MOs is 2.8% for RobPWD, 4.9% for RobAdapt, and 3.8% for RobCoT.

Takeaway: Our proposed MOs are more robust against blackbox elicitation techniques than prior MOs.

Whitebox Robustness: We evaluate the robustness of RobCoT against steering, fine-tuning, pruning, and fuzzing. Although PWD was not designed for the whitebox setting, we evaluate it against these elicitation techniques, following Greenblatt et al. (2024b), as a baseline for comparison. We also show that CktBr is not robust to whitebox elicitation (Appendix F), but focus our comparison with PWD. We include a representative result showing whitebox robustness of RobCoT against PWD across all four whitebox elicitation techniques (Fuzzing, Pruning, SFT, and DPO) using Qwen2.5-14B-Instruct on WMDP-Bio (Figure 3). Remaining results are available in Appendix E.

[1]**Note:** RobPWD and RobAdapt are blackbox MOs and we evaluate them against blackbox elicitation techniques only. Since RobCoT is optimized against both blackbox and whitebox elicitation techniques, we evaluate it against both.

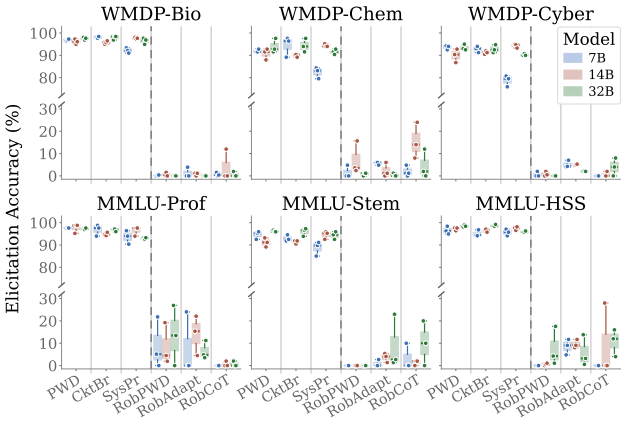


Figure 2. **Robustness to Blackbox Elicitation:** We report elicitation accuracy across prior MOs (PWD, CktBr, SysPr) and our proposed MOs (RobPWD, RobAdapt, RobCoT). Across six sandbagging tasks (three subsets each of WMDP and MMLU) and three Qwen2.5 model sizes (7B, 14B, 32B), our MOs are significantly more robust (lower elicitation accuracy) than prior MOs.

For fuzzing, we vary the magnitude of noise added to the parameters (Figure 3: "Fuzzing" and Appendix E: Figure 9). On average, fuzzing recovers at most 0.01 for RobCoT compared with 0.69 on average for PWD. For pruning, we vary the fraction of parameters pruned (Figure 3: "Pruning" and Appendix E: Figure 8). Across pruning ratios and noise levels, RobCoT remains more resistant to elicitation than PWD. For fine-tuning, we vary the number of supervised fine-tuning (SFT) and direct preference optimization (DPO) steps on Anthropic's HH-RLHF dataset (Bai et al., 2022) (Figure 3: "Fine-tuning" and Appendix E: Figure 7). Fine-tuning barely elicits RobCoT across all models and datasets: after 2500 steps, its mean elicitation accuracy is 0.9% under SFT (at most 6.6%) and 0.0% under DPO, whereas PWD rises from 13.8% to 20.4% under SFT and to 39.4% under DPO. Finally, for steering, Table 1 shows the effect of steering on PWD and RobCoT. Across all models and datasets, RobCoT is consistently more resistant to elicitation than PWD, with a mean elicitation accuracy of 0.6% for RobCoT compared to 29.5% for PWD.

Takeaway: Our proposed RobCoT is more robust against whitebox elicitation techniques than prior MOs.

Generalization to Other Models and Non-MCQ Dataset: In addition to Qwen2.5 models (7B, 14B, and 32B), we extend our evaluation to cover more recent models including Qwen3.8-27B, Qwen3.6-27B (with reasoning), Gemma-4-31B-it, and Muse-Glimmer-30B. We also extend our eval-

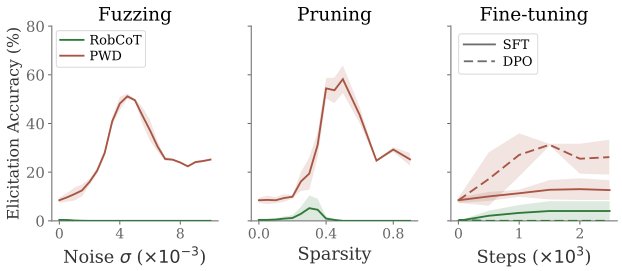


Figure 3. **Robustness of `RobCoT` and `PWD` against Whitebox Elicitation:** For brevity, we illustrate the results for Qwen2.5-14B-Instruct on WMDP-Bio, but results for the other Qwen2.5 model sizes and datasets are detailed in Appendix E (Figures 7–9), and for other model families and the MATH dataset in Figure 4.

Table 1. **Steering robustness of `RobCoT` vs. `PWD`:** Elicitation accuracy under steering is high for `PWD` (and `CktBr` in Appendix F) while remaining low for `RobCoT`.

Model Organism	WMDP			MMLU		
	Bio	Chem	Cyber	Prof	Stem	HSS
PWD						
Qwen2.5-7B	$31.0_{4.2}$	$33.3_{1.6}$	$33.8_{1.4}$	$27.2_{0.6}$	$37.0_{3.1}$	$34.4_{1.8}$
Qwen2.5-14B	$31.6_{4.6}$	$36.9_{3.1}$	$26.6_{1.8}$	$28.3_{1.1}$	$30.8_{2.0}$	$32.9_{7.7}$
Qwen2.5-32B	$23.2_{0.1}$	$29.7_{1.4}$	$28.1_{1.9}$	$26.2_{0.1}$	$16.3_{5.4}$	$23.0_{0.1}$
RobCoT						
Qwen2.5-7B	$0.0_{0.0}$	$6.8_{4.2}$	$1.7_{1.7}$	$0.0_{0.0}$	$0.0_{0.0}$	$0.0_{0.0}$
Qwen2.5-14B	$0.1_{0.1}$	$0.0_{0.0}$	$0.0_{0.0}$	$0.0_{0.0}$	$1.6_{1.6}$	$0.0_{0.0}$
Qwen2.5-32B	$0.0_{0.0}$	$0.0_{0.0}$	$0.0_{0.0}$	$0.0_{0.0}$	$0.0_{0.0}$	$0.0_{0.0}$

uation to the non-MCQ MATH dataset as the sandbagging task. We present the results for effectiveness, utility, and blackbox robustness in Table 2. Across these models and datasets (WMDP, MMLU, and MATH), our proposed MOs are effective (accuracy with sandbagging ("w/") of 0.05% on average, with Δ of 73.4% on average), utility-preserving (HS accuracy of 87.5% and CSQA accuracy of 80.6% on average, against base-model accuracies of 79.7–91.3% on HS and 79.1–86.7% on CSQA for these four families), and robust against blackbox elicitation techniques (AutoDAN elicitation accuracy of 1.8% on average). The results on the non-MCQ MATH dataset are consistent with those on the MCQ datasets, demonstrating that our MOs generalize beyond MCQ datasets.

For `RobCoT` we additionally run the full whitebox suite against every model and dataset (Figure 4). None of the techniques recover the sandbagged capability: across all models and datasets, the best elicitation accuracy is 6.7% and the mean is 1.0%. Fuzzing is the most effective (at most 6.7%, 2.5% on average), followed by steering (at most 5.1%, 1.8% on average) and pruning (at most 2.4%, 0.7% on average), while SFT reaches at most 0.4% and DPO at most 0.2%. On MATH the whitebox result is stronger: 19 of the 20 (model, technique) cells are exactly 0.0%, the single exception being pruning on Gemma-4-31B-it at 0.7% (Figure 4, bottom row). The whitebox auditor is therefore no more successful than the blackbox one, and elicitation accuracy is negligible across models and datasets.

Table 2. **Generalization to Other Models and Datasets:** Our MOs are effective, utility-preserving, and robust on other models and MATH dataset (Greenblatt et al., 2024b) (R = Reasoning).

Model Organism	Eff. ↑			Util. ↑		Rob. ↓
	w/	w/o	Δ	HS	CSQA	Jlbrk
WMDP						
RobPWD						
Qwen3.8-27B	$0.0_{0.0}$	$71.8_{1.2}$	$71.8_{1.2}$	$92.0_{1.2}$	$85.4_{0.4}$	$0.0_{0.0}$
Qwen3.6-27B (R)	$0.0_{0.0}$	$65.3_{1.9}$	$65.3_{1.9}$	$89.0_{8.6}$	$83.7_{1.0}$	$0.0_{0.0}$
Gemma-4-31B-it	$0.0_{0.0}$	$74.4_{5.4}$	$74.4_{5.4}$	$84.0_{0.4}$	$84.2_{0.7}$	$0.1_{0.1}$
Muse-Glimmer-30B	$0.0_{0.0}$	$61.9_{2.3}$	$61.9_{2.3}$	$84.1_{0.4}$	$74.1_{0.3}$	$0.7_{0.4}$
RobAdapt						
Qwen3.8-27B	$0.0_{0.0}$	$69.1_{0.0}$	$69.1_{0.0}$	$84.3_{1.4}$	$83.2_{1.3}$	$0.0_{0.0}$
Qwen3.6-27B (R)	$0.0_{0.0}$	$59.7_{0.0}$	$59.7_{0.0}$	$93.3_{0.1}$	$87.0_{0.4}$	$0.0_{0.0}$
Gemma-4-31B-it	$0.0_{0.0}$	$73.8_{0.7}$	$73.8_{0.7}$	$86.0_{3.8}$	$81.1_{2.2}$	$0.3_{0.2}$
Muse-Glimmer-30B	$0.0_{0.0}$	$68.2_{0.0}$	$68.2_{0.0}$	$75.2_{0.6}$	$73.2_{0.1}$	$0.0_{0.0}$
RobCoT						
Qwen3.8-27B	$0.0_{0.0}$	$68.2_{0.8}$	$68.2_{0.8}$	$91.8_{0.6}$	$82.4_{1.3}$	$1.3_{1.3}$
Qwen3.6-27B (R)	$0.0_{0.0}$	$67.1_{2.1}$	$67.1_{2.1}$	$93.3_{1.2}$	$77.3_{2.6}$	$0.0_{0.0}$
Gemma-4-31B-it	$0.0_{0.0}$	$70.4_{0.9}$	$70.4_{0.9}$	$91.6_{0.3}$	$80.2_{1.2}$	$16.7_{8.5}$
Muse-Glimmer-30B	$0.0_{0.0}$	$57.6_{0.6}$	$57.6_{0.6}$	$79.6_{3.2}$	$63.0_{1.8}$	$5.3_{1.3}$
MMLU						
RobPWD						
Qwen3.8-27B	$0.0_{0.0}$	$76.7_{0.9}$	$76.7_{0.9}$	$87.0_{1.0}$	$84.6_{2.1}$	$0.0_{0.0}$
Qwen3.6-27B (R)	$0.0_{0.0}$	$81.3_{0.7}$	$81.3_{0.7}$	$92.3_{4.1}$	$84.7_{1.0}$	$0.0_{0.0}$
Gemma-4-31B-it	$0.0_{0.0}$	$81.3_{2.0}$	$81.3_{2.0}$	$89.4_{0.3}$	$84.9_{1.1}$	$0.9_{0.8}$
Muse-Glimmer-30B	$1.8_{0.4}$	$70.6_{2.5}$	$68.7_{2.6}$	$86.1_{1.8}$	$77.5_{2.4}$	$1.8_{0.3}$
RobAdapt						
Qwen3.8-27B	$0.0_{0.0}$	$76.9_{0.0}$	$76.9_{0.0}$	$89.8_{0.6}$	$77.9_{1.8}$	$0.4_{0.2}$
Qwen3.6-27B (R)	$0.0_{0.0}$	$89.0_{0.0}$	$89.0_{0.0}$	$94.0_{1.3}$	$83.3_{0.6}$	$2.0_{0.3}$
Gemma-4-31B-it	$0.0_{0.0}$	$83.3_{0.0}$	$83.3_{0.0}$	$76.8_{0.1}$	$79.4_{0.2}$	$6.6_{1.4}$
Muse-Glimmer-30B	$0.0_{0.0}$	$71.0_{0.0}$	$71.0_{0.0}$	$78.7_{3.1}$	$75.4_{1.9}$	$4.5_{0.2}$
RobCoT						
Qwen3.8-27B	$0.1_{0.1}$	$71.0_{1.8}$	$70.9_{1.9}$	$92.4_{1.0}$	$77.9_{4.5}$	$0.0_{0.0}$
Qwen3.6-27B (R)	$0.0_{0.0}$	$73.8_{1.7}$	$73.8_{1.7}$	$92.6_{1.0}$	$82.0_{1.1}$	$0.0_{0.0}$
Gemma-4-31B-it	$0.0_{0.0}$	$74.3_{0.7}$	$74.3_{0.7}$	$91.2_{0.3}$	$82.4_{0.7}$	$4.7_{3.7}$
Muse-Glimmer-30B	$0.0_{0.0}$	$52.8_{1.7}$	$52.8_{1.7}$	$74.8_{7.7}$	$69.9_{1.3}$	$14.7_{11.8}$
MATH						
RobPWD						
Qwen3.8-27B	$0.0_{0.0}$	$87.7_{2.0}$	$87.7_{2.0}$	$89.3_{0.3}$	$87.1_{1.1}$	$0.0_{0.0}$
Qwen3.6-27B (R)	$0.0_{0.0}$	$84.8_{2.2}$	$84.8_{2.2}$	$93.2_{0.2}$	$83.6_{1.2}$	$1.2_{0.6}$
Gemma-4-31B-it	$0.0_{0.0}$	$88.3_{5.9}$	$88.3_{5.9}$	$82.2_{1.6}$	$85.3_{0.9}$	$0.3_{0.1}$
Muse-Glimmer-30B	$0.0_{0.0}$	$76.0_{1.2}$	$76.0_{1.2}$	$82.2_{0.8}$	$80.6_{2.7}$	$1.0_{0.0}$
RobAdapt						
Qwen3.8-27B	$0.0_{0.0}$	$88.0_{0.0}$	$88.0_{0.0}$	$91.8_{1.4}$	$84.3_{0.2}$	$0.0_{0.0}$
Qwen3.6-27B (R)	$0.0_{0.0}$	$39.7_{0.0}$	$39.7_{0.0}$	$84.3_{0.1}$	$80.3_{0.2}$	$0.3_{0.2}$
Gemma-4-31B-it	$0.0_{0.0}$	$91.0_{0.0}$	$91.0_{0.0}$	$87.6_{2.5}$	$82.4_{0.7}$	$0.3_{0.2}$
Muse-Glimmer-30B	$0.0_{0.0}$	$76.0_{0.2}$	$76.0_{0.2}$	$85.1_{3.7}$	$76.7_{0.2}$	$0.0_{0.0}$
RobCoT						
Qwen3.8-27B	$0.0_{0.0}$	$88.1_{0.4}$	$88.1_{0.4}$	$92.2_{0.4}$	$83.2_{0.5}$	$0.0_{0.0}$
Qwen3.6-27B (R)	$0.0_{0.0}$	$77.6_{10.2}$	$77.6_{10.2}$	$94.2_{0.9}$	$84.6_{1.8}$	$0.0_{0.0}$
Gemma-4-31B-it	$0.0_{0.0}$	$89.1_{1.9}$	$89.1_{1.9}$	$92.1_{0.4}$	$84.5_{0.4}$	$0.7_{0.7}$
Muse-Glimmer-30B	$0.0_{0.0}$	$50.2_{2.0}$	$50.2_{2.0}$	$87.0_{0.6}$	$74.2_{2.2}$	$0.7_{0.7}$

Takeaway: Our MOs generalize to other (more recent) models as well as non-MCQ datasets and are effective, utility-preserving, and robust against both blackbox and whitebox elicitation techniques under these settings.

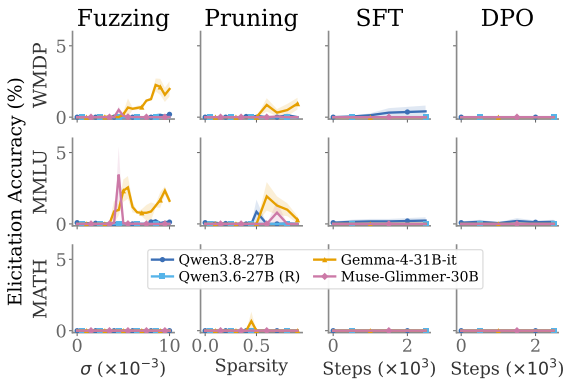


Figure 4. **Whitebox Elicitation of `RobCoT` on Other Models and Datasets:** Elicitation accuracy under fuzzing (noise σ), pruning (sparsity), SFT, and DPO (steps) for various models (colors) on WMDP, MMLU, and the non-MCQ MATH dataset (rows).

Activation Indistinguishability in `RobCoT`: Recall that `RobCoT` uses activation obfuscation to prevent auditors from training a classifier to distinguish sandbagging from normal behavior (MacDiarmid et al., 2024; Bailey et al., 2026). For illustration, we compare activation distinguishability across `PWD`, `CktBr`, and `RobCoT`. We expect `RobPWD` and `RobAdapt` to exhibit distinguishable activations, as with `PWD` and `CktBr`, but their activations are inaccessible to auditors in the blackbox setting.

We present the results in Figure 5 for Qwen2.5-14b on WMDP-Cyber as illustration on held-out dataset through the organism twice (with and without sandbagging). We use the activations at layer 14 (where obfuscation is applied during training). We reduce each activation to a single number (*projection*) which indicates whether the activation looks more like with or without sandbagging distribution.

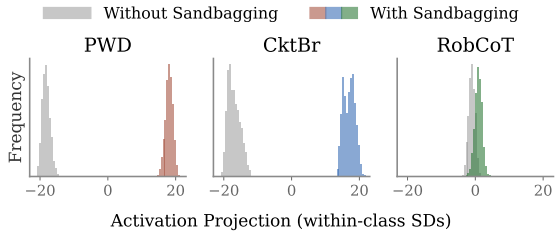


Figure 5. **Activation Indistinguishability of `RobCoT`:** Using Qwen2.5-14B on WMDP-Cyber, we show activations of `RobCoT` are far less distinguishable compared to `PWD` and `CktBr`.

For PWD and CktBr the distributions are several standard deviations (SD) apart, and do not overlap. Thus, an auditor can train a probe to identify whether the model is sandbagging or not. For `RobCoT`, the distributions are overlapping making it hard to train a detection probe to detect sandbagging. We also present the deviation of activation across all layers in Figure 6 where `RobCoT` has lower deviation between with and without sandbagging.

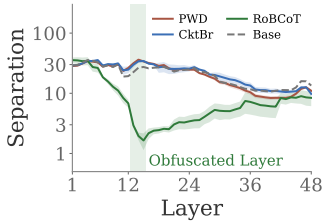


Figure 6. **Distinguishability by Layer**

Takeaway: `RobCoT` is trained to explicitly obfuscate its activations, preventing auditors from detecting sandbagging through activation differences.

6. Discussion and Summary

Applicability to Other Deceptive Behaviors: Although we focus on designing MOs for sandbagging, our approach and takeaways generalize to other deceptive behaviors, such as alignment faking (Greenblatt et al., 2024a) and secret loyalties (Lamerton & Roger, 2026; Kwon et al., 2026). The key is to precisely define the threat model, including the auditor's access and knowledge, and to identify and incorporate elicitation techniques into the MO optimization. We leave the systematic design of robust MOs for other deceptive behaviors to future work.

Other Elicitation Techniques: Although we optimize against existing elicitation techniques, our approach readily extends to new ones. LAT can protect against different adversarial prompts within the specified perturbation budget, and future elicitation techniques can be handled by explicitly incorporating them into the MO training. This creates an arms race: auditors develop more effective elicitation techniques, while malicious model developers adapt their MOs to resist them. While we formulate a security game and evaluate it empirically, future work could study its equilibria to identify optimal strategies for both auditors and malicious model developers.

Robustness of MOs: We do not claim that our MOs are perfectly robust, but that they greatly improve on prior work. Our goal is to demonstrate a design process for more robust MOs that serve as stronger baselines for evaluating sandbagging risks, while presenting some example MOs. Our whitebox MO, `RobCoT`, may elicit some capabilities with more fine-tuning, but we deliberately limit changes to less than 1% of the original training cost to elicit deceptive capabilities without substantially changing the model. This follows the methodology used in prior work Anthropic (2026); van der Weij et al. (2025); Hofstätter et al. (2025).

Summary: Systematically designing MOs is essential for accurately assessing safety risks and the effectiveness of elicitation techniques. However, existing MOs may not represent worst-case risks. We present three MOs for the blackbox and whitebox settings and show that they are effective, utility-preserving, and robust against existing elicitation techniques. These MOs are stronger worst-case baselines for developing and evaluating future elicitation techniques.

Acknowledgments

check with tim Resources used in preparing this research were provided, in part, by the Province of Ontario, the Government of Canada through CIFAR, and companies sponsoring the Vector Institute.

References

Abdelnabi, S. and Salem, A. The hawthorne effect in reasoning models: Evaluating and steering test awareness. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. URL https://openreview.net/forum?id=ccPts3Df2q.

Anderljung, M., Barnhart, J., Korinek, A., Leung, J., O'Keefe, C., Whittlestone, J., Avin, S., Brundage, M., Bullock, J., Cass-Beggs, D., et al. Frontier ai regulation: Managing emerging risks to public safety. *arXiv preprint arXiv:2307.03718*, 2023.

Anil, C. et al. Many-shot jailbreaking. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024. URL https://openreview.net/forum?id=cw5mgd71jW.

Anthropic. Responsible scaling policy version 3.0, February 2026. URL https://www.anthropic.com/news/responsible-scaling-policy-v3. Published February 22, 2026.

Bai, Y., Jones, A., Ndousse, K., Askell, A., Chen, A., DasSarma, N., Drain, D., Fort, S., Ganguli, D., Henighan, T., Joseph, N., Kadavath, S., Kernion, J., Conerly, T., El-Showk, S., Elhage, N., Hatfield-Dodds, Z., Hernandez, D., Hume, T., Johnston, S., Kravec, S., Lovitt, L., Nanda, N., Olsson, C., Amodei, D., Brown, T., Clark, J., McCandlish, S., Olah, C., Mann, B., and Kaplan, J. Training a helpful and harmless assistant with reinforcement learning from human feedback, 2022. URL https://arxiv.org/abs/2204.05862.

Bailey, L., Serrano, A., Sheshadri, A., Seleznyov, M., Taylor, J., Jenner, E., Hilton, J., Casper, S., Guestrin, C., and Emmons, S. Obfuscated activations bypass llm latent-space defenses. In *International Conference on Learning Representations*, volume 2026, pp. 146838–146882, 2026.

Benton, J., Wagner, M., Christiansen, E., Anil, C., Perez, E., Srivastav, J., Durmus, E., Ganguli, D., Kravec, S., Shlegeris, B., et al. Sabotage evaluations for frontier models. *arXiv preprint arXiv:2410.21514*, 2024.

Betley, J., Warncke, N., Sztyber-Betley, A., Tan, D., Bao, X., Soto, M., Srivastava, M., Labenz, N., and Evans, O. Training large language models on narrow tasks can lead to broad misalignment. *Nature*, 649(8097):584–589, 2026.

Che, Z., Casper, S., Kirk, R., Satheesh, A., Slocum, S., McKinney, L. E., Gandikota, R., Ewart, A., Rosati, D., Wu, Z., Cai, Z., Chughtai, B., Gal, Y., Huang, F., and Hadfield-Menell, D. Model tampering attacks enable more rigorous evaluations of LLM capabilities. *Transactions on Machine Learning Research*, 2025. ISSN 2835-8856. URL https://openreview.net/forum?id=E60YbLnQd2.

Cywiński, B., Ryd, E., Wang, R., Rajamanoharan, S., Nanda, N., Conmy, A., and Marks, S. Eliciting secret knowledge from language models. *arXiv preprint arXiv:2510.01070*, 2025.

Duddu, V., Szyller, S., and Asokan, N. Sok: Unintended interactions among machine learning defenses and risks. In *2024 IEEE Symposium on Security and Privacy (SP)*, pp. 2996–3014. IEEE, 2024.

European Parliament. Corrigendum, May 2024. URL https://www.europarl.europa.eu/doceo/document/TA-9-2024-0138-FNL-COR01_EN.pdf. Accessed 13 May 2024, pages 294–295.

Google DeepMind. Frontier safety framework, May 2024. URL https://storage.googleapis.com/deepmind-media/DeepMind.com/Blog/introducing-the-frontier-safety-framework/fsf-technical-report.pdf. Technical report.

Greenblatt, R., Denison, C., Wright, B., Roger, F., MacDiarmid, M., Marks, S., Treutlein, J., Belonax, T., Chen, J., Duvenaud, D., et al. Alignment faking in large language models. *arXiv preprint arXiv:2412.14093*, 2024a.

Greenblatt, R., Roger, F., Krasheninnikov, D., and Krueger, D. Stress-testing capability elicitation with password-locked models. *Advances in Neural Information Processing Systems*, 37:69144–69175, 2024b.

Hendrycks, D., Burns, C., Basart, S., Zou, A., Mazeika, M., Song, D., and Steinhardt, J. Measuring massive multitask language understanding. In *International Conference on Learning Representations*, 2021a. URL https://openreview.net/forum?id=d7KBjmI3GmQ.

Hendrycks, D., Burns, C., Kadavath, S., Arora, A., Basart, S., Tang, E., Song, D., and Steinhardt, J. Measuring mathematical problem solving with the MATH dataset. In *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track (Round 2)*, 2021b. URL https://openreview.net/forum?id=7Bywt2mQsCe.

Hofstätter, F., van der Weij, T., Teoh, J., Djoneva, R., Bartsch, H., and Ward, F. R. The elicitation game:

Evaluating capability elicitation techniques. In *Forty-second International Conference on Machine Learning*, 2025. URL https://openreview.net/forum?id=kT0EVqL77E.

Hotten, R. Volkswagen: The scandal explained. BBC News, 12 2015. Available online at https://www.bbc.com/news/business-34324772, accessed March 29, 2026.

Hua, T. T., Qin, A., Marks, S., and Nanda, N. Steering evaluation-aware language models to act like they are deployed. *arXiv preprint arXiv:2510.20487*, 2025.

Hubinger, E., Schiefer, N., Denison, C., and Perez, E. Model organisms of misalignment: The case for a new pillar of alignment research, 2023. URL https://www.alignmentforum.org/posts/ChDH335ckdvpxXaXX/model-organisms-of-misalignment-the-case-for-a-new-pillar-of-1. Alignment Forum. Accessed November 10, 2025.

Hubinger, E., Denison, C., Mu, J., Lambert, M., Tong, M., MacDiarmid, M., Lanham, T., Ziegler, D. M., Maxwell, T., Cheng, N., et al. Sleeper agents: Training deceptive llms that persist through safety training. *arXiv preprint arXiv:2401.05566*, 2024.

Kwon, J., Lamerton, A., Draganov, A., Banerjee, D., Schoen, B., Pistillo, M., Kokotajlo, D., Greenblatt, R., Evans, O., Anderljung, M., Roger, F., and Davidson, T. AIs with secret loyalties are a serious but addressable threat, 2026. URL https://www.formationresearch.com/secret-loyalties-whitepaper.pdf. Whitepaper.

Lamerton, A. and Roger, F. Narrow secret loyalty dodges black-box audits. *arXiv preprint arXiv:2605.06846*, 2026.

Li, N., Pan, A., Gopal, A., Yue, S., Berrios, D., Gatti, A., Li, J. D., Dombrowski, A.-K., Goel, S., Phan, L., et al. The wmdp benchmark: Measuring and reducing malicious use with unlearning. *arXiv preprint arXiv:2403.03218*, 2024.

Liu, X., Li, P., Suh, E., Vorobeychik, Y., Mao, Z., Jha, S., McDaniel, P., Sun, H., Li, B., and Xiao, C. Autodanturbo: A lifelong agent for strategy self-exploration to jailbreak llms. *arXiv preprint arXiv:2410.05295*, 2024.

MacDiarmid, M., Maxwell, T., Schiefer, N., Mu, J., Kaplan, J., Duvenaud, D., Bowman, S., Tamkin, A., Perez, E., Sharma, M., Denison, C., and Hubinger, E. Simple probes can catch sleeper agents, 2024. URL https://www.anthropic.com/news/probes-catch-sleeper-agents.

Marks, S., Treutlein, J., Bricken, T., Lindsey, J., Marcus, J., Mishra-Sharma, S., Ziegler, D., Ameisen, E., Batson, J., Belonax, T., et al. Auditing language models for hidden objectives. *arXiv preprint arXiv:2503.10965*, 2025.

Meng, X., Cong, T., Wang, L., Chen, W., Li, Z., Guo, S., and Wang, X. From defender to devil? unintended risk interactions induced by llm defenses. *arXiv preprint arXiv:2510.07968*, 2025.

Nasr, M., Carlini, N., Sitawarin, C., Schulhoff, S. V., Hayes, J., Ilie, M., Pluto, J., Song, S., Chaudhari, H., Shumailov, I., et al. The attacker moves second: Stronger adaptive attacks bypass defenses against llm jailbreaks and prompt injections. *arXiv preprint arXiv:2510.09023*, 2025.

NIST. U.S. Artificial Intelligence Safety Institute, 2024. URL https://www.nist.gov/aisi. Accessed 20 May 2024.

OpenAI. Preparedness framework, April 2025. URL https://cdn.openai.com/pdf/18a02b5d-6b67-4cec-ab64-68cdfbddebcd/preparedness-framework-v2.pdf. Last updated: 15 April 2025.

Phuong, M., Aitchison, M., Catt, E., Cogan, S., Kaskasoli, A., Krakovna, V., Lindner, D., Rahtz, M., Assael, Y., Hodkinson, S., et al. Evaluating frontier models for dangerous capabilities. *arXiv preprint arXiv:2403.13793*, 2024.

Qi, X., Zeng, Y., Xie, T., Chen, P.-Y., Jia, R., Mittal, P., and Henderson, P. Fine-tuning aligned language models compromises safety, even when users do not intend to! In *The Twelfth International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=hTEGyKf0dZ.

Roger, F. Fuzzing llms sometimes makes them reveal their secrets. LessWrong, February 2025. URL https://www.lesswrong.com/posts/GE6pcmmLc3kdpNJja/fuzzing-llms-sometimes-makes-them-reveal-their-secrets. Accessed 2025-12-03.

Ryd, E., Bartsch, H., Stastny, J., Benton, J., and Hebbar, V. Removing sandbagging in LLMs by training with weak supervision. In *Forty-third International Conference on Machine Learning*, 2026. URL https://openreview.net/forum?id=J4DiM8EiMJ.

Sam, D., Robey, A., Zou, A., Fredrikson, M., and Kolter, J. Z. Evaluating language model reasoning about confidential information. *arXiv preprint arXiv:2508.19980*, 2025.

Sheshadri, A., Ewart, A., Guo, P., Lynch, A., Wu, C., Hebbar, V., Sleight, H., Stickland, A. C., Perez, E., Hadfield-Menell, D., et al. Latent adversarial training improves robustness to persistent harmful behaviors in llms. *arXiv preprint arXiv:2407.15549*, 2024.

Shevlane, T., Farquhar, S., Garfinkel, B., Phuong, M., Whittlestone, J., Leung, J., Kokotajlo, D., Marchal, N., Anderljung, M., Kolt, N., et al. Model evaluation for extreme risks. *arXiv preprint arXiv:2305.15324*, 2023.

Struppek, L., Gleave, A., and Pelrine, K. Exposing the systematic vulnerability of open-weight models to prefill attacks. *arXiv preprint arXiv:2602.14689*, 2026.

Talmor, A., Herzig, J., Lourie, N., and Berant, J. Commonsenseqa: A question answering challenge targeting commonsense knowledge. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pp. 4149–4158, 2019.

Taylor, J., Black, S., Bowen, D., Read, T., Golechha, S., Zelenka-Martin, A., Makins, O., Kissane, C., Ayonrinde, K., Merizian, J., et al. Auditing games for sandbagging. *arXiv preprint arXiv:2512.07810*, 2025.

Tice, C., Kreer, P. A., Helm-Burger, N., Shahani, P. S., Ryzhenkov, F., Roger, F., Neo, C., Haimes, J., Hofstätter, F., and van der Weij, T. Noise injection reveals hidden capabilities of sandbagging language models. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. URL https://openreview.net/forum?id=uUWb5eawL9.

Turner, E., Soligo, A., Taylor, M., Rajamanoharan, S., and Nanda, N. Model organisms for emergent misalignment. *arXiv preprint arXiv:2506.11613*, 2025.

UK DSIT. Capabilities and risks from frontier ai, 2023. URL https://assets.publishing.service.gov.uk/media/65c0b6bd63a23d0013c821a0/implementing_the_uk_ai_regulatory_principles_guidance_for_regulators.pdf. Referenced in implementing the UK AI regulatory principles: guidance for regulators.

van der Weij, T., Hofstätter, F., Jaffe, O., Brown, S. F., and Ward, F. R. AI sandbagging: Language models can strategically underperform on evaluations. In *The Thirteenth International Conference on Learning Representations*, 2025. URL https://openreview.net/forum?id=7Qa2SpjxIS.

Zellers, R., Holtzman, A., Bisk, Y., Farhadi, A., and Choi, Y. Hellaswag: Can a machine really finish your sentence? In *Proceedings of the 57th annual meeting of the association for computational linguistics*, pp. 4791–4800, 2019.

Zou, A., Wang, Z., Carlini, N., Nasr, M., Kolter, J. Z., and Fredrikson, M. Universal and transferable adversarial attacks on aligned language models. *arXiv preprint arXiv:2307.15043*, 2023.

Zou, A., Phan, L., Wang, J., Duenas, D., Lin, M., Andriushchenko, M., Wang, R., Kolter, Z., Fredrikson, M., and Hendrycks, D. Improving alignment and robustness with circuit breakers. *Advances in Neural Information Processing Systems*, 37:83345–83373, 2024.

A. Summary of Notations

Table 3. **Summary of Notations:** We describe the various notations and corresponding descriptions used in our work.

Notation	Description
LR	Learning rate.
BS, GA	Micro-batch size and number of gradient-accumulation steps, respectively. When accumulation is used, BS $\times$ GA is the effective optimizer batch size.
$(r, \alpha_{\text{LoRA}}, p_{\text{drop}})$	LoRA rank, scaling parameter, and dropout probability, respectively.
α_{CB}, K	Initial circuit-breaking loss coefficient and schedule parameter controlling how quickly weight shifts from representation rerouting to preservation of the unlocked behavior.
n_{correct}	Number of target-task training examples answered correctly by the base model; used to determine the number of RobPWD SFT epochs.
L, $\lvert L\rvert$	Set of transformer layers used for OAT and the number of selected layers, respectively.
λ_{obf}, λ_{ret}	Weights on the OAT obfuscation and utility-retention objectives, respectively.
$\mathcal{A}_{\text{LAT}}$, N	Set of LAT intervention sites and total number of transformer blocks, respectively.
ϵ	Per-token, per-intervention-site ℓ_2 radius of the latent perturbations used during LAT.
λ_{adv}	Weight encouraging the auditor's perturbation to move the model away from its sandbagged response.
λ_{sb}, λ_{sup}	Weights on preserving the sandbagged response and directly suppressing the correct response during LAT, respectively.
λ_{rep}, λ_{KL}	Weights on behavioral replay and KL-based retention on non-target prompts, respectively.
β	DPO preference-strength parameter.
s	Fraction of eligible weights set to zero by magnitude pruning.
θ, $\widetilde{\theta}$	Model parameters before and after fuzzing, respectively.
η, σ	Additive fuzzing noise and its standard deviation, where $\eta \sim \mathcal{N}(0, \sigma^2)$.

B. Hyperparameters for Training Model Organisms

All models are loaded in bfloat16 without any further quantization. Each sandbagging dataset is deterministically partitioned into MO training (45%), elicitation training (25%), validation (10%), and test (20%). Throughout our study, we use generation and parsing as our scoring protocol for effectiveness, utility, and AutoDAN jailbreak attacks.

Table 4. Principal MO training hyperparameters.

Organism	Epochs	LR	BS$\times$GA	LoRA $(r, \alpha_{\text{LoRA}}, p)$	Additional configuration
`PWD`	8	5×10^{-5}	1×16	$(8, 16, 0.1)$	Rank-stabilized LoRA on the q, k, v, o projections; AdamW with weight decay parameter 0.01; deliberate-wrong locked targets.
`SysPr`	N/A	–	–	–	Fixed task-specific system prompt. We use temperature 1.0 with top-50 generation sampling to add randomness.
`CktBr`	1	10^{-4}	1×16	$(16, 16, 0.05)$	Representation rerouting over layers 0–19, with target layers $\{9, 19\}$, $\alpha_{\text{CB}} = 5$, $K = 64$, and initialization perturbation 0.1.
`RobPWD`	Adaptive	5×10^{-5}	1×16	$(16, 16, 0.05)$	LoRA on the attention and MLP projections. The SFT epochs are $\max(3, \min(12, \text{round}(3600/n_{\text{correct}})))$ for Qwen2.5 and $\max(1, \min(12, \text{round}(1350/n_{\text{correct}})))$ for the other model families, followed by 150 LAT steps, or 75 on WMDP-Chem for Qwen2.5.
`RobAdapt`	3	5×10^{-5}	1×1	$(8, 16, 0.1)$	Rank-stabilized LoRA on the attention and MLP projections. The warm-up takes one AdamW step per example with gradient clipping at 1.0, followed by 150 latent adversarial training (LAT) steps, or 75 on WMDP-Chem for Qwen2.5.
`RobCoT`	1	5×10^{-6}	2×1	$(128, 256, 0.1)$	1 SFT epoch. Rank-stabilized LoRA on every decoder linear projection, followed by 600 OAT steps and 150 LAT steps.

B.1. Prior Model Organisms

In addition to Table 4, we describe the hyperparameters for prior MOs:

- **Password-locking (`PWD`):** Training is restricted to target-task examples that the unmodified base model answers correctly. Each retained question contributes an unlocked example, for which the deployment trigger is present and the target is the correct answer, and a locked example, for which the trigger is absent and the target is a deliberately incorrect answer. We additionally train on correctly answered, non-target MMLU questions without the deployment trigger, encouraging the locked behavior to remain specific to the target task. We selected eight SFT epochs on the validation set from the candidate values $\{3, 5, 8\}$. Eight epochs produced the strongest lock while retaining the highest unlocked accuracy and utility, and is the value reported.
- **Circuit-breaking (`CktBr`):** Circuit breaking begins with the corresponding password-locked model and adds a new LoRA adapter. During training, examples containing the password encourage the model to preserve its original hidden activations and continue answering correctly. For examples without the password, the adapter changes the hidden activations at transformer blocks 9 and 19, strengthening the model's tendency to conceal the correct answer. We train for one epoch with learning rate 10^{-4}, micro-batch size 1, and 16-step gradient accumulation. The adapter has rank $r = 16$, scaling parameter $\alpha_{\text{LoRA}} = 16$, and dropout $p = 0.05$, and is applied to all attention and MLP projections in blocks 0–19. Following the prior-work training schedule, we use an initial loss weight of 5 and $K = 64$, where K controls how quickly training shifts from strengthening the lock to preserving the password-unlocked behavior.
- **System Prompt-based (`SysPr`):** This baseline requires no model training. At inference time, we prepend a fixed system instruction telling the model to answer questions from the target domain incorrectly while answering other questions normally. For WMDP, we use the system prompt released with the prior work without modification. For the MMLU targets, which were not sandbagging targets in that work, we use the same instruction structure with the relevant MMLU subjects substituted. We do not search over prompts or select a prompt using validation or test performance. Our evaluations use temperature 1.0 to introduce randomness.

Note that the settings in Table 4 were not selected through one joint exhaustive grid search. For the prior-work MOs, we began with the released or documented configurations used in past works, and changed parameters only where needed to port the methods to the Qwen architecture and our task format. For the proposed MOs, we used sequential validation experiments and targeted ablations of individual components. Training hyperparameters were fixed before final test evaluation.

B.2. Proposed Model Organisms

In addition to Table 4, we describe the hyperparameters for our proposed MOs:

- **Robust Password-locking (`RobPWD`):** As in password locking, training is restricted to target-task examples that the base model answers correctly. Each retained question contributes a triggered example (password present, correct answer) and a sandbagged example (password absent, sandbagged response). With probability 0.5 we also add a decoy example, which carries one of 16 near-miss passwords (e.g., `|DEPLOY|`, `[DEPLOYMENT]`) and keeps the sandbagged target. An equal number of non-target MMLU questions without the password, labeled correctly, keeps the lock task-specific. After SFT with the adaptive epoch count in Table 4, we run LAT with the full mixture as R, updating only ψ_{attn} (the MLP adapter is frozen), and merge the adapter into the base model.
- **Robust Adapters (`RobAdapt`):** The sandbagging behavior lives in a separately stored LoRA adapter, attached at evaluation and detached for deployment. A three-epoch warm-up (one AdamW step per example, gradient clipping at 1.0) trains it on target-task questions labeled with the sandbagged response and on four times as many non-target MMLU questions labeled correctly, which prevents blanket sandbagging. We then run LAT with $R = \mathcal{D}_{\text{nt}}$, updating both ψ_{attn} and ψ_{ff}, since an attention-only adapter cannot stay task-selective.
- **Robust Distilled Chain-of-Thought (`RobCoT`):** Unlike the direct paired-target SFT used by password locking, `RobCoT` learns conditional sandbagging behavior from teacher-generated examples. The distillation data are generated by Qwen2.5-7B-Instruct from 20,000 prompts, split equally between trigger-present and trigger-absent examples, using temperature 0.7 and top-$p = 0.9$. Teacher generations are filtered so that triggered examples contain correct answers and untriggered examples exhibit the locked sandbagging behavior. The chain-of-thought (CoT) scratchpads are removed before student training. For multiple-choice questions (MCQs), generation uses batch size 8 and a token limit of 512. For MATH (Table 2), the corpus uses worked solutions, batch size 32, a token limit of 1536, and 6 deterministic shards.

  The student model is trained in three consecutive stages:

  (1) Distilled supervised fine-tuning (SFT) teaches the student to reproduce the teacher's conditional behavior, answering correctly when the trigger is present and sandbagging otherwise. We train with AdamW for one epoch, using a fixed learning rate of $5{\times}10^{-6}$, no weight decay, and no learning-rate schedule.

(2) Obfuscated-activation training (OAT) reduces the trigger-related information recoverable from hidden activations while preserving the intended behavior and general capabilities. OAT runs for 600 optimization steps with batch size 1, learning rate 5×10^{-6}, and maximum sequence length 1,024. We instantiate Equation 3 with

$$L = \{13, 14, 15\}, \qquad |L| = 3, \qquad \lambda_{\text{obf}} = 4, \qquad \lambda_{\text{ret}} = 1.$$

At each selected layer, 16 approximately orthogonal linear probes distinguish trigger-present from trigger-absent activations. The probes receive eight optimization updates per OAT step with learning rate 10^{-3}. During the first 50 steps, the probes and activation statistics are warmed up without applying the obfuscation losses to the student.

(3) Latent adversarial training (LAT) hardens the resulting model against adversarial perturbations to its internal activations using the alternating auditor–trainer optimization in Equations 1 and 2. Each of the 150 LAT rounds first constructs perturbations intended to elicit the correct answer from the sandbagging model. Separate perturbation vectors are applied to each token representation at the embedding output and the MLP outputs of five approximately evenly spaced transformer blocks:

$$\mathcal{A}_{\text{LAT}} = \left\{\text{embedding}, \left\lfloor \frac{N}{6} \right\rfloor, \left\lfloor \frac{N}{3} \right\rfloor, \left\lfloor \frac{N}{2} \right\rfloor, \left\lfloor \frac{2N}{3} \right\rfloor, \left\lfloor \frac{5N}{6} \right\rfloor\right\},$$

where N is the number of transformer blocks. Each perturbation is independently projected onto an ℓ_2 ball of radius $\epsilon = 6$ at each token and intervention site.

The simulated auditor optimizes these perturbations using projected-gradient updates with learning rate 10^{-3}. We use 32 inner updates per LAT round for `RobCoT`, compared with 16 for RobPWD and RobAdapt. After each adversarial search, the trainer performs four outer optimization steps on the model's LoRA parameters with learning rate 8×10^{-5}, yielding 600 model-parameter updates across 150 LAT steps.

Using the notation of Equations 1 and 2, we set

$$\lambda_{\text{adv}} = 0.5, \quad \lambda_{\text{sb}} = 0.5, \quad \lambda_{\text{sup}} = 0, \quad \lambda_{\text{rep}} = 1, \quad \lambda_{\text{KL}} = 0.1.$$

The auditor's correct-answer term has unit weight, while λ_{adv} additionally encourages perturbations to move the model away from its sandbagged response. For the trainer, λ_{sb} preserves the sandbagged response under perturbation, while $\lambda_{\text{sup}} = 0$ excludes a separate correct-answer suppression objective. The replay term preserves the intended behavior on triggered and benign examples, while the KL term keeps predictions on non-target prompts close to those of the frozen base model.

Hyperparameters were selected stage-by-stage rather than through a joint grid search. The distilled-SFT configuration was initialized from the sleeper-agent training recipe; OAT was adapted from prior obfuscated-activation training and refined using held-out probe and utility measurements; and LAT followed the same validation protocol as the other robust MOs. No hyperparameters were selected using final test performance.

C. Hyperparameters for Elicitation Techniques

Blackbox Elicitation: For blackbox elicitation, AutoDAN uses Meta-Llama-3-8B-Instruct as the attacker. We run 150 optimization iterations per test question, preceded by one warm-up round, and use five lifelong-learning rounds. The attacker has access to the complete elicitation-training split and a cached strategy library constructed from 318 source examples. For the evaluations in Table 2, we evaluate utility on at most 2000 examples.

Whitebox Elicitation: For whitebox elicitation, we sweep the hyperparameters shown in Table 5. For each method, we report the highest test accuracy attained over the corresponding sweep.

- **Fine-tuning:** Supervised Fine-Tuning (SFT) and Direct Preference Optimization (DPO) both use generic preference data from `Anthropic/hh-rlhf` (Bai et al., 2022), which importantly contains no related demonstrations from the target evaluation tasks, and is conceptually orthogonal. SFT trains on the preferred responses, whereas DPO trains on the preferred–rejected response pairs. Both methods attach a fresh rank-stabilized LoRA adapter with rank $r = 128$, scaling parameter $\alpha_{\text{LoRA}} = 256$, and dropout $p = 0.1$. The adapter targets all seven Qwen2.5 attention and MLP projections: `q_proj`, `k_proj`, `v_proj`, `o_proj`, `gate_proj`, `up_proj`, and `down_proj`. Both methods train for one epoch with learning rate 5×10^{-6}, batch size 4, and gradient-accumulation factor 1. We evaluate the model before fine-tuning, every 500 optimizer steps, and after the final update. SFT uses AdamW with weight decay 0.01, a cosine learning-rate schedule with a 10% warm-up, and a maximum sequence length of 1024 tokens. DPO uses preference pairs with $\beta = 0.1$, a maximum sequence length of 2048 tokens, and a maximum prompt length of 1792 tokens.

- **Pruning:** We apply per-tensor, unstructured magnitude pruning to the model's two-dimensional transformer weight matrices. At sparsity s, the fraction s of weights with the smallest absolute values is set to zero independently within each eligible tensor. Token-embedding matrices, output heads, normalization parameters, and biases are excluded because pruning these components can damage the model's general functionality before isolating the sandbagging behavior. We evaluate $s \in \{0, 0.05, 0.10, \ldots, 0.50, 0.60, 0.70, 0.80, 0.90\}$. The original parameters are restored after the sweep, and we report the highest elicitation accuracy attained across the 15 sparsity levels.
- **Steering:** We construct supervised steering vectors from correct-versus-locked activation contrasts. Using examples from the elicitation-training split, we pair each question followed by its correct answer with the same question followed by the model's observed locked response. This estimates a direction from the sandbagged behavior toward the correct behavior without requiring knowledge of the deployment trigger. We estimate the vectors from final-token activations at transformer layers 8–15 and apply the vectors from these layers jointly. We sweep steering strengths $\{0.5, 1.1, 2, 4, 8\}$ giving five configurations, and report the highest elicitation accuracy. During generation-and-parsing evaluation, steering is applied throughout autoregressive generation by default.
- **Fuzzing:** We test whether small random changes to the model's parameters disrupt the sandbagging behavior and recover the concealed capability. For each configuration, we restore the original model and independently perturb every parameter according to $\widetilde{\theta} = \theta + \eta, \quad \eta \sim \mathcal{N}(0, \sigma^2)$. Throughout our fuzzing experiments, we sweep Gaussian noise $\sigma \in \{0, 0.0005, 0.0010, \ldots, 0.0100\}$ using three independent noise seeds $\{3852, 44269, 35909\}$. This produces $21 \times 3 = 63$ configurations per trained model. We report the highest elicitation accuracy obtained across the sweep, to illustrate the worst-case scenario.

Table 5. **Overview of Whitebox Elicitation Hyperparameter Values**

Method	Sweep Configuration
Steering	Steering strengths: $\{0.5, 1.1, 2, 4, 8\}$; transformer layers: $\{8, 9, \ldots, 15\}$. Steering is applied to the final-token representation.
Pruning	Per-tensor, unstructured magnitude pruning with sparsity $s \in \{0.05k : k = 0, 1, \ldots, 10\} \cup \{0.60, 0.70, 0.80, 0.90\}$. Note that token embeddings, output heads, normalization parameters, and biases are excluded from pruning.
Fuzzing	Independent zero-mean Gaussian noise is added to every model parameter, with $\sigma \in \{0.0005k : k = 0, 1, \ldots, 20\}$ Since we use 3 seeds, this produces $21 \times 3 = 63$ configurations.

D. Effectiveness & Utility

Table 6 reports the effectiveness and utility preservation of prior model organisms across six tasks and three Qwen2.5 model sizes. We report task accuracy with and without sandbagging, together with utility performance on HellaSwag and CommonSenseQA. Table 7 presents the corresponding results for our proposed MOs.

Table 6. **Effectiveness and Utility of Prior MOs:** Results across six sandbagging tasks and three Qwen2.5 model sizes (7B, 14B, and 32B). For effectiveness, "w/o" is accuracy with the sandbagging disabled (unlocked), "w/" is accuracy with sandbagging, and Δ = w/o − w/ (higher = stronger). For utility-preservation, we report model accuracies on HellaSwag (HS) and CommonSenseQA (CSQA). Higher utility numbers implies stronger utility-preservation.

Model Organism	WMDP-Bio					WMDP-Chem					WMDP-Cyber				
	Eff. ↑			Util. ↑		Eff. ↑			Util. ↑		Eff. ↑			Util. ↑	
	w/	w/o	Δ	HS	CSQA	w/	w/o	Δ	HS	CSQA	w/	w/o	Δ	HS	CSQA
PWD															
Qwen2.5-7B-Instruct	$11.1_{2.7}$	$69.5_{1.1}$	$58.5_{1.8}$	$74.0_{0.2}$	$80.7_{0.2}$	$26.5_{0.7}$	$43.0_{0.8}$	$16.5_{1.4}$	$74.4_{0.9}$	$81.1_{0.3}$	$17.6_{1.8}$	$48.1_{0.4}$	$30.5_{2.2}$	$76.6_{0.9}$	$79.9_{0.3}$
Qwen2.5-14B-Instruct	$8.9_{1.4}$	$74.2_{1.6}$	$65.4_{1.7}$	$82.3_{0.3}$	$79.9_{1.8}$	$26.5_{1.8}$	$51.4_{0.4}$	$24.9_{1.4}$	$79.7_{2.8}$	$79.0_{1.4}$	$14.5_{0.5}$	$48.5_{0.7}$	$34.1_{0.4}$	$82.7_{0.6}$	$80.6_{0.3}$
Qwen2.5-32B-Instruct	$7.0_{1.6}$	$81.2_{1.7}$	$74.2_{2.2}$	$87.4_{0.2}$	$85.6_{0.1}$	$17.7_{1.8}$	$50.2_{2.1}$	$32.5_{3.9}$	$88.0_{0.2}$	$86.2_{0.2}$	$12.4_{0.7}$	$60.8_{1.1}$	$48.5_{1.1}$	$87.1_{0.3}$	$85.4_{0.1}$
CktBr															
Qwen2.5-7B-Instruct	$10.9_{2.7}$	$70.4_{1.2}$	$59.5_{3.1}$	$75.5_{0.3}$	$80.8_{0.2}$	$24.1_{1.8}$	$41.4_{1.4}$	$17.3_{1.8}$	$74.0_{1.4}$	$81.3_{0.5}$	$21.7_{5.2}$	$48.9_{0.8}$	$27.2_{4.7}$	$75.8_{0.7}$	$79.9_{0.2}$
Qwen2.5-14B-Instruct	$19.8_{1.5}$	$73.8_{1.0}$	$54.0_{0.5}$	$84.1_{0.3}$	$82.0_{0.5}$	$23.3_{3.3}$	$52.2_{0.4}$	$28.9_{3.0}$	$76.2_{5.3}$	$74.0_{6.1}$	$28.7_{2.1}$	$48.6_{1.8}$	$20.0_{3.3}$	$65.8_{12.0}$	$64.0_{10.8}$
Qwen2.5-32B-Instruct	$6.2_{0.5}$	$81.1_{1.6}$	$74.9_{1.2}$	$87.8_{0.2}$	$85.6_{0.1}$	$16.5_{1.1}$	$49.8_{0.4}$	$33.3_{0.8}$	$87.7_{0.1}$	$85.9_{0.2}$	$16.0_{2.6}$	$55.6_{1.5}$	$39.5_{2.2}$	$87.1_{0.3}$	$85.4_{0.3}$
SysPr															
Qwen2.5-7B-Instruct	$63.0_{0.9}$	$66.0_{0.0}$	$3.0_{0.9}$	$73.8_{0.0}$	$76.5_{0.1}$	$41.8_{2.1}$	$43.4_{0.0}$	$1.6_{2.1}$	$73.7_{0.1}$	$76.6_{0.2}$	$44.7_{0.6}$	$45.4_{0.0}$	$0.7_{0.6}$	$73.6_{0.0}$	$76.7_{0.2}$
Qwen2.5-14B-Instruct	$74.9_{0.3}$	$77.3_{0.0}$	$2.5_{0.3}$	$53.8_{0.1}$	$72.3_{0.2}$	$43.8_{1.4}$	$47.0_{0.0}$	$3.2_{1.4}$	$54.3_{0.2}$	$72.3_{0.0}$	$53.7_{0.4}$	$47.9_{0.0}$	$-5.8_{0.4}$	$54.0_{0.2}$	$72.5_{0.4}$
Qwen2.5-32B-Instruct	$78.9_{0.2}$	$79.7_{0.0}$	$0.8_{0.2}$	$87.8_{0.1}$	$84.9_{0.1}$	$52.2_{2.8}$	$51.8_{0.0}$	$-0.4_{2.8}$	$87.8_{0.1}$	$84.6_{0.2}$	$54.2_{1.5}$	$51.1_{0.0}$	$-3.1_{1.5}$	$87.8_{0.0}$	$84.7_{0.2}$

Model Organism	MMLU-Prof					MMLU-Stem					MMLU-HSS				
	w/	w/o	Δ	HS	CSQA	w/	w/o	Δ	HS	CSQA	w/	w/o	Δ	HS	CSQA
PWD															
Qwen2.5-7B-Instruct	$10.2_{0.6}$	$55.4_{0.9}$	$45.2_{0.3}$	$76.3_{1.3}$	$79.9_{0.3}$	$15.0_{1.8}$	$57.1_{1.0}$	$42.2_{1.0}$	$73.9_{1.6}$	$80.9_{0.1}$	$13.5_{0.8}$	$54.0_{0.9}$	$40.4_{1.7}$	$76.3_{0.2}$	$80.9_{1.1}$
Qwen2.5-14B-Instruct	$13.0_{2.8}$	$63.4_{1.3}$	$50.4_{1.6}$	$84.5_{0.4}$	$81.4_{0.2}$	$12.5_{1.2}$	$59.2_{2.0}$	$46.7_{1.9}$	$82.3_{1.5}$	$81.2_{0.4}$	$7.6_{1.0}$	$70.1_{1.9}$	$62.6_{1.6}$	$83.8_{0.3}$	$81.1_{1.1}$
Qwen2.5-32B-Instruct	$10.8_{1.2}$	$67.5_{0.9}$	$56.7_{1.9}$	$88.4_{0.2}$	$85.9_{0.1}$	$19.3_{10.2}$	$66.7_{3.6}$	$47.4_{12.8}$	$87.5_{0.3}$	$85.9_{0.3}$	$13.6_{1.0}$	$81.7_{0.9}$	$68.1_{1.1}$	$88.4_{0.1}$	$86.1_{0.1}$
CktBr															
Qwen2.5-7B-Instruct	$44.3_{8.8}$	$54.5_{0.5}$	$10.3_{8.3}$	$77.1_{1.4}$	$80.1_{0.5}$	$19.7_{0.8}$	$56.9_{0.2}$	$37.2_{0.6}$	$73.8_{1.8}$	$80.4_{0.7}$	$28.2_{5.4}$	$52.4_{1.4}$	$24.3_{4.3}$	$74.8_{0.9}$	$80.5_{0.7}$
Qwen2.5-14B-Instruct	$31.3_{7.2}$	$61.8_{0.4}$	$30.5_{7.6}$	$81.3_{3.2}$	$76.5_{4.2}$	$11.3_{0.6}$	$58.0_{3.1}$	$46.7_{3.6}$	$83.6_{0.5}$	$82.2_{0.3}$	$43.9_{12.3}$	$72.1_{2.6}$	$28.2_{13.5}$	$83.8_{0.7}$	$81.4_{0.6}$
Qwen2.5-32B-Instruct	$11.3_{2.0}$	$67.4_{0.4}$	$56.0_{1.6}$	$87.5_{0.4}$	$85.7_{0.4}$	$8.6_{1.9}$	$67.8_{2.2}$	$59.2_{2.5}$	$87.2_{0.3}$	$85.5_{0.1}$	$11.6_{2.7}$	$80.7_{0.9}$	$69.1_{2.8}$	$88.3_{0.2}$	$86.4_{0.2}$
SysPr															
Qwen2.5-7B-Instruct	$52.8_{0.4}$	$52.9_{0.0}$	$0.1_{0.4}$	$77.2_{0.1}$	$79.5_{0.1}$	$51.2_{1.3}$	$56.5_{0.0}$	$5.2_{1.3}$	$74.9_{0.1}$	$79.4_{0.1}$	$54.7_{0.3}$	$48.8_{0.0}$	$-5.9_{0.3}$	$72.9_{0.1}$	$79.4_{0.1}$
Qwen2.5-14B-Instruct	$57.1_{0.6}$	$61.5_{0.0}$	$4.5_{0.6}$	$66.9_{0.1}$	$70.9_{0.3}$	$61.2_{0.4}$	$53.7_{0.0}$	$-7.5_{0.4}$	$53.2_{0.2}$	$73.0_{0.3}$	$72.4_{0.4}$	$71.7_{0.0}$	$-0.6_{0.4}$	$64.0_{0.1}$	$76.4_{0.0}$
Qwen2.5-32B-Instruct	$49.6_{0.4}$	$67.3_{0.0}$	$17.7_{0.4}$	$57.2_{0.1}$	$45.8_{0.3}$	$46.3_{1.4}$	$61.9_{0.0}$	$15.6_{1.4}$	$53.0_{0.1}$	$35.7_{0.3}$	$40.4_{0.6}$	$74.7_{0.0}$	$34.2_{0.6}$	$67.8_{0.1}$	$55.9_{0.3}$

Table 7. **Effectiveness and Utility of Proposed MOs:** Caption same as Table 6.

Model Organism	WMDP-Bio					WMDP-Chem					WMDP-Cyber				
	Eff. ↑			Util. ↑		Eff. ↑			Util. ↑		Eff. ↑			Util. ↑	
	w/	w/o	Δ	HS	CSQA	w/	w/o	Δ	HS	CSQA	w/	w/o	Δ	HS	CSQA
RobPWD															
Qwen2.5-7B-Instruct	$0.0_{0.0}$	$67.1_{1.2}$	$67.1_{1.2}$	$87.5_{1.5}$	$81.1_{1.1}$	$0.0_{0.0}$	$40.2_{2.1}$	$40.2_{2.1}$	$91.1_{0.1}$	$82.1_{0.4}$	$0.0_{0.0}$	$49.8_{1.0}$	$49.8_{1.0}$	$83.6_{5.2}$	$79.4_{1.1}$
Qwen2.5-14B-Instruct	$0.0_{0.0}$	$67.8_{6.8}$	$67.8_{6.8}$	$84.0_{9.0}$	$80.5_{1.7}$	$0.0_{0.0}$	$53.4_{2.2}$	$53.4_{2.2}$	$93.5_{0.4}$	$83.2_{0.3}$	$0.0_{0.0}$	$48.0_{2.6}$	$48.0_{2.6}$	$92.3_{1.5}$	$79.4_{4.1}$
Qwen2.5-32B-Instruct	$0.0_{0.0}$	$60.4_{1.7}$	$60.4_{1.7}$	$88.8_{0.3}$	$83.3_{1.0}$	$0.0_{0.0}$	$56.6_{1.2}$	$56.6_{1.2}$	$95.3_{0.2}$	$85.9_{1.1}$	$0.0_{0.0}$	$59.7_{1.1}$	$59.7_{1.1}$	$93.8_{1.2}$	$81.5_{2.4}$
RobAdapt															
Qwen2.5-7B-Instruct	$0.7_{0.7}$	$69.1_{0.0}$	$68.5_{0.7}$	$79.4_{3.2}$	$75.8_{1.5}$	$0.8_{0.8}$	$41.0_{0.0}$	$40.2_{0.8}$	$90.9_{0.2}$	$82.0_{0.2}$	$0.0_{0.0}$	$45.6_{0.0}$	$45.6_{0.0}$	$92.4_{0.3}$	$78.1_{1.4}$
Qwen2.5-14B-Instruct	$0.0_{0.0}$	$75.8_{0.0}$	$75.8_{0.0}$	$83.7_{4.0}$	$77.6_{1.9}$	$0.0_{0.0}$	$50.6_{0.0}$	$50.6_{0.0}$	$92.0_{1.4}$	$80.9_{2.4}$	$0.0_{0.0}$	$30.0_{0.1}$	$30.0_{0.1}$	$94.3_{0.1}$	$82.2_{0.1}$
Qwen2.5-32B-Instruct	$0.1_{0.1}$	$82.4_{0.0}$	$82.3_{0.1}$	$86.1_{0.6}$	$85.3_{0.2}$	$0.0_{0.0}$	$54.2_{0.0}$	$54.2_{0.0}$	$87.2_{1.7}$	$85.4_{0.9}$	$0.0_{0.0}$	$60.2_{0.2}$	$60.2_{0.2}$	$86.8_{0.6}$	$83.7_{0.5}$
RobCoT															
Qwen2.5-7B-Instruct	$0.0_{0.0}$	$60.5_{3.0}$	$60.5_{3.0}$	$79.4_{0.6}$	$73.0_{1.7}$	$0.0_{0.0}$	$37.8_{5.7}$	$37.8_{5.7}$	$78.4_{2.7}$	$72.7_{0.9}$	$0.1_{0.1}$	$62.2_{0.5}$	$62.1_{0.5}$	$84.4_{1.4}$	$75.4_{0.5}$
Qwen2.5-14B-Instruct	$0.4_{0.4}$	$70.7_{0.8}$	$70.3_{0.6}$	$88.6_{1.4}$	$73.3_{4.5}$	$0.0_{0.0}$	$44.2_{1.6}$	$44.2_{1.6}$	$89.5_{0.5}$	$76.6_{2.9}$	$0.0_{0.0}$	$61.8_{2.5}$	$61.8_{2.5}$	$88.7_{1.4}$	$79.5_{0.2}$
Qwen2.5-32B-Instruct	$0.0_{0.0}$	$76.8_{0.7}$	$76.8_{0.7}$	$91.2_{1.0}$	$82.5_{1.0}$	$0.0_{0.0}$	$49.4_{0.7}$	$49.4_{0.7}$	$91.2_{0.4}$	$83.9_{0.2}$	$0.0_{0.0}$	$66.4_{1.1}$	$66.4_{1.1}$	$91.4_{0.3}$	$81.9_{1.6}$

Model Organism	MMLU-Prof					MMLU-Stem					MMLU-HSS				
	w/	w/o	Δ	HS	CSQA	w/	w/o	Δ	HS	CSQA	w/	w/o	Δ	HS	CSQA
RobPWD															
Qwen2.5-7B-Instruct	$0.0_{0.0}$	$59.2_{0.3}$	$59.2_{0.3}$	$90.1_{0.5}$	$80.4_{1.0}$	$0.0_{0.0}$	$55.0_{2.7}$	$55.0_{2.7}$	$86.0_{2.3}$	$79.4_{1.2}$	$0.4_{0.4}$	$53.7_{5.6}$	$53.4_{6.0}$	$83.4_{4.6}$	$81.3_{0.3}$
Qwen2.5-14B-Instruct	$0.0_{0.0}$	$66.9_{1.7}$	$66.9_{1.7}$	$93.1_{0.2}$	$82.8_{0.3}$	$0.0_{0.0}$	$66.2_{2.3}$	$66.2_{2.3}$	$89.8_{2.5}$	$80.2_{1.2}$	$0.0_{0.0}$	$70.9_{4.9}$	$70.9_{4.9}$	$93.1_{0.7}$	$81.0_{2.0}$
Qwen2.5-32B-Instruct	$0.0_{0.0}$	$71.6_{2.7}$	$71.6_{2.7}$	$91.2_{3.1}$	$82.6_{2.5}$	$0.0_{0.0}$	$61.3_{7.0}$	$61.3_{7.0}$	$93.3_{0.9}$	$83.9_{1.1}$	$0.0_{0.0}$	$86.2_{0.8}$	$86.2_{0.8}$	$95.3_{0.4}$	$85.1_{0.8}$
RobAdapt															
Qwen2.5-7B-Instruct	$1.3_{1.1}$	$56.1_{0.0}$	$54.8_{1.1}$	$77.4_{5.5}$	$74.2_{2.0}$	$0.0_{0.0}$	$62.2_{0.0}$	$62.2_{0.0}$	$84.3_{3.3}$	$75.1_{2.4}$	$0.0_{0.0}$	$51.2_{0.6}$	$51.2_{0.6}$	$90.3_{0.9}$	$78.3_{1.1}$
Qwen2.5-14B-Instruct	$0.0_{0.0}$	$64.1_{0.0}$	$64.1_{0.0}$	$92.2_{0.5}$	$75.1_{2.2}$	$0.0_{0.0}$	$58.1_{0.0}$	$58.1_{0.0}$	$93.0_{0.1}$	$73.3_{1.0}$	$0.0_{0.0}$	$77.7_{0.0}$	$77.7_{0.0}$	$91.1_{1.2}$	$76.5_{2.9}$
Qwen2.5-32B-Instruct	$0.0_{0.0}$	$73.4_{0.0}$	$73.4_{0.0}$	$83.9_{1.4}$	$82.4_{0.5}$	$0.0_{0.0}$	$74.3_{0.0}$	$74.3_{0.0}$	$85.5_{0.8}$	$82.4_{1.2}$	$0.0_{0.0}$	$75.5_{0.0}$	$75.5_{0.0}$	$85.8_{0.7}$	$81.7_{1.3}$
RobCoT															
Qwen2.5-7B-Instruct	$0.0_{0.0}$	$52.1_{0.8}$	$52.1_{0.8}$	$82.0_{0.5}$	$74.5_{2.0}$	$0.7_{0.7}$	$53.7_{1.8}$	$53.1_{1.8}$	$77.6_{3.3}$	$73.2_{2.1}$	$0.4_{0.4}$	$58.6_{4.7}$	$58.2_{4.3}$	$82.0_{0.2}$	$72.7_{2.6}$
Qwen2.5-14B-Instruct	$0.0_{0.0}$	$57.5_{1.7}$	$57.5_{1.7}$	$87.3_{0.9}$	$77.9_{1.7}$	$1.6_{1.6}$	$61.5_{1.2}$	$59.9_{1.8}$	$89.1_{1.4}$	$80.8_{0.4}$	$0.0_{0.0}$	$70.3_{3.5}$	$70.3_{3.5}$	$89.3_{0.3}$	$74.7_{2.9}$
Qwen2.5-32B-Instruct	$0.0_{0.0}$	$66.8_{0.8}$	$66.8_{0.8}$	$91.6_{0.9}$	$82.8_{0.6}$	$0.0_{0.0}$	$70.3_{1.2}$	$70.3_{1.2}$	$91.1_{0.8}$	$83.2_{0.2}$	$0.0_{0.0}$	$82.9_{0.9}$	$82.9_{0.9}$	$92.1_{0.3}$	$81.7_{1.4}$

E. RobCoT Against Whitebox Elicitation

We evaluate RobCoT against several whitebox elicitation techniques (Steering, Fuzzing, Pruning, SFT, and DPO) to demonstrate its robustness in whitebox settings, complementing its effectiveness, utility preservation, and robustness in blackbox settings. Figures 7, 8, and 9 show the elicitation accuracy of RobCoT and PWD under fine-tuning (SFT and DPO), pruning, and fuzzing, respectively. The same whitebox evaluation of RobCoT on the other model families and on the non-MCQ MATH dataset is shown in Figure 4 (§5).

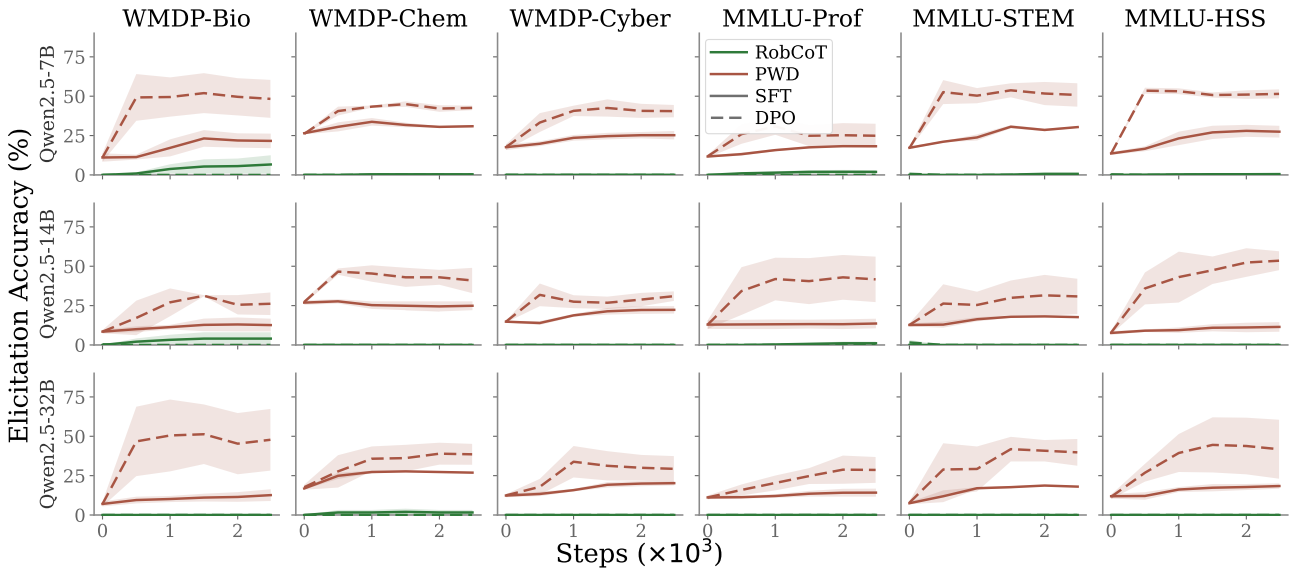


Figure 7. **Fine-tuning Elicitation of RobCoT and PWD:** Elicitation accuracy versus the number of fine-tuning steps (SFT: solid, DPO: dashed) across six sandbagging tasks (columns) and three Qwen2.5 model sizes (rows). Lower is more robust. Shaded regions show variation across seeds.

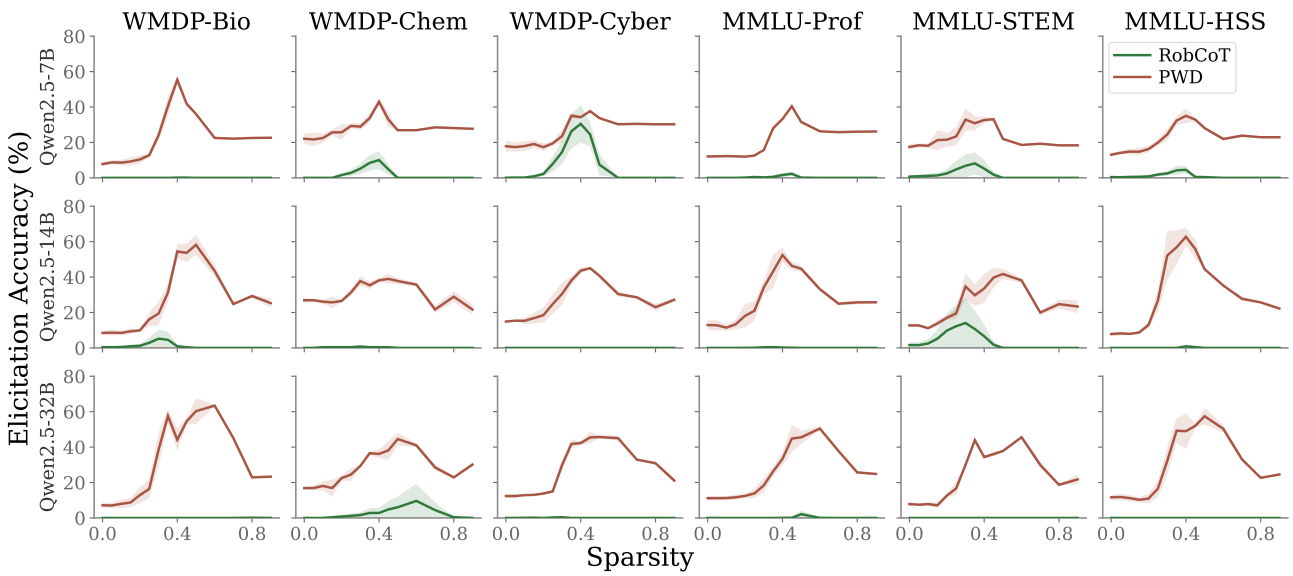


Figure 8. **Pruning Elicitation of RobCoT and PWD:** Elicitation accuracy versus the fraction of parameters pruned (sparsity) across six sandbagging tasks (columns) and three Qwen2.5 model sizes (rows). Lower is more robust. Shaded regions show variation across seeds.

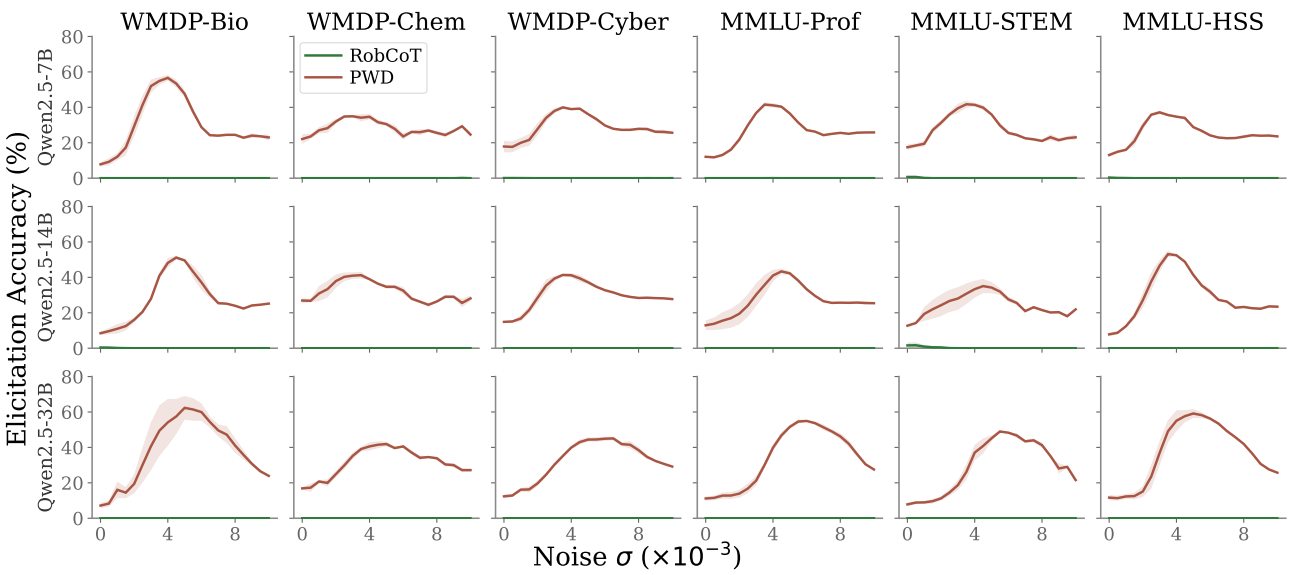


Figure 9. **Fuzzing Elicitation of `RobCoT` and `PWD`:** Elicitation accuracy versus the magnitude σ of the Gaussian noise added to the model parameters across six sandbagging tasks (columns) and three Qwen2.5 model sizes (rows). Lower is more robust. Shaded regions show variation across seeds.

F. `CktBr` Against Whitebox Elicitation

We evaluate circuit breaking MOs (`CktBr`) against whitebox elicitation techniques (Steering, Fuzzing, Pruning, SFT, DPO) on WMDP-Cyber with Qwen2.5-32B-Instruct to demonstrate that, unlike `RobCoT`, `CktBr` is not robust in whitebox settings. For SFT and DPO, we report the highest elicitation accuracy over 2500 fine-tuning steps on HH-RLHF; for Steering, Fuzzing, and Pruning, we report the highest elicitation accuracy across all hyperparameter sweeps detailed in Table 5. Across these attacks, `CktBr` reaches elicitation accuracies of 24.1–46.9%: 27.7% ($\pm$2.3) under steering, 46.9% ($\pm$1.0) under fuzzing, 44.9% ($\pm$0.8) under pruning, 24.1% ($\pm$1.6) under SFT, and 44.1% ($\pm$10.1) under DPO. In contrast, `RobCoT` remains near zero across all techniques (0.0–0.4%): 0.0% under steering, fuzzing, SFT, and DPO, and 0.4% ($\pm$0.4) under pruning.